分 类 号_____ 学号_____
学校代码_____ 密级_____

華中科技大學

硕士学位论文

(学术型□ 专业型☑)

基于驱动程序替换的物联网设备固件仿真技术研究

学位申请人：

学 科 专 业：

指 导 教 师：

答 辩 日 期：

A Dissertation Submitted in Partial Fulfillment of the Requirements for

Research on IoT Device Firmware Emulation Technology Based on Driver Replacement

Candidate :

Major :

Supervisor :

Huazhong University of Science and Technology

Wuhan 430074, P. R. China

March, 2026

独创性声明

本人声明所呈交的学位论文是我个人在导师的指导下进行的研究工作及取得的研究成果。尽我所知，除文中已标明引用的内容外，本论文不包含任何其他人或集体已经发表或撰写过的研究成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期：　　年　　月　　日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权华中科技大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本论文属于 保　密 □，在 ____ 年解密后适用本授权书。
不保密 □。

（请在以上方框内打“√”）

学位论文作者签名：　　指导教师签名：

日期：　　年　　月　　日　　日期：　　年　　月　　日

摘　　要

随着物联网技术的快速发展，物联网设备数量呈现爆炸式增长，其固件安全问题日益突出。固件仿真技术是进行固件安全测试和漏洞挖掘的重要手段，然而现有的固件仿真方法存在外设建模成本高、硬件依赖强、自动化程度不足等问题。传统的基于硬件建模的方法需要为每种外设手动建立仿真模型，工作量大且难以覆盖多样化的硬件生态；基于驱动层替换的方法虽然避免了底层硬件建模，但驱动函数的定位与替换仍高度依赖人工干预，难以实现规模化应用。

针对上述问题，本文提出一种结合大语言模型的自动化固件仿真方法。该方法的核心思想是通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型的代码理解与生成能力自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量，从而在无需手动建立外设模型的情况下实现固件在仿真环境中的稳定运行。

本文的主要工作包括三个方面。首先，设计并实现了面向嵌入式固件的硬件依赖代码自动识别模块，该模块通过 MMIO 模式识别、数据流追踪与递归归纳技术，从固件中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界，构建可查询的驱动知识库。其次，构建了基于大语言模型的驱动函数分析与替换模块，该模块利用大语言模型理解驱动函数的语义，通过结构化提示词模板和函数分类策略，自动生成可在仿真环境中运行的替代代码，并通过编译修复循环保障代码的可编译性。最后，引入了动态测试反馈闭环机制，当固件在仿真环境中运行出现异常时，系统能够自动分析异常原因并反馈给驱动替换模块，通过”生成—测试—反馈—修正”的闭环流程持续优化仿真质量。

实验结果表明，本文方法能够有效支持多种嵌入式平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）的固件仿真，显著降低了固件仿真的技术门槛和人工成本，为后续的模糊测试和漏洞挖掘奠定了基础。

关键词：物联网安全；固件仿真；驱动程序替换；大语言模型；静态分析

Abstract

With the rapid development of Internet of Things (IoT) technology, the number of IoT devices has grown explosively, and firmware security issues have become increasingly prominent. Firmware emulation is an important approach for firmware security testing and vulnerability discovery. However, existing firmware emulation methods face challenges such as high peripheral modeling costs, strong hardware dependencies, and insufficient automation. Traditional hardware modeling-based methods require manually building simulation models for each peripheral, which is labor-intensive and difficult to cover the diverse hardware ecosystem. Although driver layer replacement-based methods avoid low-level hardware modeling, the identification and replacement of driver functions still heavily rely on manual intervention, making it difficult to achieve large-scale application.

To address these issues, this paper proposes an automated firmware emulation method combined with large language models (LLMs). The core idea is to automatically identify hardware-dependent code in firmware through static analysis, use the code understanding and generation capabilities of LLMs to automatically generate alternative driver implementations, and build a dynamic testing feedback loop to iteratively optimize emulation quality, thereby enabling firmware to run stably in emulation environments without manually building peripheral models.

The main contributions of this paper include three aspects. First, a hardware dependency code automatic identification module for embedded firmware is designed and implemented, which automatically extracts hardware-related function call points, peripheral register access patterns, and driver function boundaries from firmware through MMIO pattern recognition, data flow tracking, and recursive induction techniques, constructing a queryable driver knowledge base. Second, a driver function analysis and replacement module based on LLMs is built, which utilizes LLMs to understand driver function semantics, automatically

generates alternative code that can run in emulation environments through structured prompt templates and function classification strategies, and ensures code compilability through a compilation repair loop. Finally, a dynamic testing feedback loop mechanism is introduced. When firmware runs abnormally in the emulation environment, the system can automatically analyze the cause of the abnormality and feed it back to the driver replacement module, continuously optimizing emulation quality through a ”generate-test-feedback-revise” closed-loop process.

Experimental results show that the proposed method can effectively support firmware emulation for multiple embedded platforms (STM32, NXP) and operating systems (FreeRTOS, RT-Thread, bare metal), significantly reducing the technical barrier and manual cost of firmware emulation, laying the foundation for subsequent fuzzing and vulnerability discovery.

Keywords: IoT Security; Firmware Emulation; Driver Replacement; Large Language Model; Static Analysis

目　　录

摘　要 I
Abstract II
1　绪论
1.1　研究背景与意义 ..... (1)
1.2　国内外研究现状 ..... (3)
1.3　研究内容与主要贡献 ..... (10)
1.4　论文组织结构 ..... (10)
2　背景知识与相关技术
2.1　物联网设备硬件基础 ..... (12)
2.2　物联网操作系统与驱动 ..... (17)
2.3　大模型代码分析智能体框架 ..... (20)
2.4　程序分析技术 ..... (28)
2.5　固件仿真与模糊测试 ..... (30)
2.6　本章小结 ..... (34)
3　系统设计与实现
3.1　整体架构设计 ..... (35)
3.2　驱动函数接口静态分析与识别模块 ..... (41)
3.3　驱动函数分析、替换与编译模块 ..... (48)
3.4　动态测试反馈闭环模块 ..... (60)
3.5　本章小结 ..... (66)
4　实验设计与结果分析
4.1　实验设计 ..... (68)
4.2　静态分析模块评估 ..... (73)
4.3　LLM 驱动替换模块评估 ..... (74)
4.4　仿真环境稳定性评估 ..... (76)
4.5　与现有方法的对比 ..... (77)
4.6　模糊测试应用验证 ..... (81)
4.7　本章小结 ..... (82)

5 总结与展望

5.1 工作总结 ..... (83)

5.2 论文创新点 ..... (84)

5.3 论文贡献 ..... (85)

5.4 局限性与不足 ..... (85)

5.5 未来工作展望 ..... (86)

致 谢

参考文献 ..... (89)

攻读硕士学位期间发表的学术论文 ..... (92)

1 绪论

1.1 研究背景与意义

近年来，随着物联网（Internet of Things, IoT）技术迅速发展，物联网设备已被广泛用于智能家居、智慧城市、新能源、车联网、医疗健康等多个应用领域。根据GSMA 发布的《The mobile economy 2020》报告显示，2019 年全球物联网总连接数达到 120 亿，预计到 2025 年，全球物联网总连接数规模将达到 246 亿，年复合增长率高达 13%[1]。物联网操作系统是一种面向物联网设备和应用的轻量级嵌入式操作系统，它可以提供更好的设备管理、数据处理和安全性能。目前，市场上有许多物联网操作系统，例如华为的 OpenHarmony[?]、亚马逊的 FreeRTOS[3]、Linux 基金会的 Zephyr[4] 等。然而，物联网和嵌入式操作系统面临着严峻的安全挑战[5]，如恶意攻击、数据泄露、隐私侵犯等，这些安全问题可能会给用户带来损失和危害。在真实世界中发生的物联网安全问题也层出不穷。2018 年，一款名为”Mirai” 的恶意软件[6]利用了 IoT 设备的弱点，通过攻击嵌入式设备的操作系统，将其变成僵尸网络的一部分，对互联网上的服务器发起了大规模的 DDoS 攻击。俄乌冲突期间，乌克兰遭遇了俄罗斯发起的破坏性网络攻击，该攻击利用一种名为 Industroyer 的恶意软件控制电力系统中的开关和断路器，意图瘫痪乌克兰的电力系统。这些安全事件的不断发生表明，当前物联网和嵌入式操作系统安全问题的研究刻不容缓。

固件仿真（Firmware Re-Hosting）技术是一种克服固件分析硬件依赖问题的有效方法，它通过在商用计算机和服务器的软件模拟器上运行嵌入式系统固件，实现对嵌入式设备的软件级模拟，使得包含嵌入式系统的固件无需实际硬件即可进行安全测试[7]。然而，当前固件仿真技术的**自动化程度仍然严重不足**，这主要体现在以下几个方面：首先，现有的外设仿真方法大多需要人工编写模型或基于有限规则的自动推断。硬件在环方案需要为每种外设手动配置物理设备连接；基于原始硬件的外设建模方案需要人工分析数据手册并编写仿真代码；基于程序分析的方案虽然能自动推断部分外设行为，但推断规则有限，难以覆盖复杂外设的状态机逻辑。其次，

驱动层替换方案（如 HALucinator）虽然利用了微控制器制造商提供的硬件抽象层（HAL），通过函数 Hook 的方式进行替换，但驱动函数的定位、语义理解和替换代码生成仍然需要大量人工干预。分析人员需要手动识别固件中哪些部分存在硬件依赖并需要替换，理解驱动函数与硬件的交互逻辑，并编写相应的替代代码，这个过程不仅耗时耗力，而且容易出错。这种**自动化程度不足导致的人工成本高昂**，严重制约了固件仿真技术在大规模安全测试中的应用，成为当前物联网设备安全领域亟待解决的关键问题。

现有固件仿真技术的自动化不足主要体现在三个核心环节。在驱动函数定位环节，识别固件中哪些部分存在硬件依赖并需要替换是一项具有挑战性的任务。在嵌入式程序中，硬件依赖代码往往分散在各个模块中，通过函数调用、全局变量访问、寄存器操作等多种形式存在，手动定位这些代码需要分析人员对固件结构和硬件特性有深入理解，过程既耗时又容易出错。在外设建模环节，当前的自动化方法只能处理简单的外设行为，对于网络控制器、USB 设备等具有多通道、DMA 传输、中断嵌套等特性的复杂外设，往往需要人工编写仿真模型。在替换代码生成环节，现有工具主要通过函数 Hook 返回预设值或执行简化逻辑，缺乏对驱动函数内部语义的深入理解，无法准确模拟硬件行为，需要人工编写复杂的替代代码。这三个环节的自动化不足相互叠加，使得固件仿真过程对分析人员经验的依赖程度过高，难以适应大规模、多样化的固件安全测试需求。

针对上述问题，本研究致力于提升固件仿真技术的自动化程度，降低人工成本，实现高效的物联网设备固件安全测试。具体而言，本研究提出一种基于驱动程序替换的固件仿真方法，通过静态分析技术自动识别固件中的硬件依赖代码，利用大语言模型的代码理解能力深入分析驱动函数语义并自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量。该方法的核心创新在于将自动化技术贯穿于驱动函数定位、语义分析和代码生成的全过程，显著减少了对分析人员人工干预的依赖。通过提升固件仿真的自动化水平，本研究不仅能够降低固件安全测试的成本，还能够提高测试效率和覆盖率，为物联网设备的大规模安全测试提供技术支撑，具有重要的理论意义和应用价值。

1.2 国内外研究现状

固件是嵌入式设备中的软件组成部分，通常包含了设备的操作系统、驱动程序、应用程序和配置文件等。固件的安全性对于保护设备和用户数据的完整性和隐私至关重要。然而，由于固件的复杂性、多样性和闭源性，对固件进行有效的安全测试是一项具有挑战性的任务。

基于固件仿真的模糊测试技术在近年来受到了越来越多的关注和研究，出现了一些代表性的工具和系统，如 FIRMADYNE[7]、FUZZWARE[8] 等。这些工具和系统在不同程度上实现了对固件仿真和模糊测试的支持，并在实际的固件中发现了一些新的漏洞。然而，基于固件仿真的模糊测试技术仍然存在一些挑战和问题，如仿真环境的准确性、测试用例的有效性等，需要进一步的改进和优化。下面将从固件仿真技术、嵌入式设备模糊测试技术两个方面阐述当前基于固件仿真的嵌入式设备固件模糊测试技术研究现状。

1.2.1 固件仿真技术研究现状

当前固件仿真的主要挑战是为模拟器中没有实现的硬件提供有效的输入。诸如 QEMU[9] 之类的仿真器提供了模拟各种 CPU 体系结构的能力，但对商用嵌入式系统中使用的各种各样的外设，例如计时器、UART、以太网控制器等，提供的支持却十分有限，而这些外设对于正确实现固件仿真却是十分必要的。

固件仿真技术的发展经历了从依赖真实硬件到全自动化模拟的演进过程。早期研究采用硬件在环（Hardware in the Loop, HIL）的方法，如 Zaddich 等人提出的 Avatar[10] 和 Muench 等人提出的 Avatar2[11]，通过混合执行的方式将固件分析过程中对外设的访问转发到物理硬件中。然而，这种方案仍然依赖实际设备，且仿真环境和物理设备之间的切换影响了运行速度。为摆脱对真实硬件的依赖，研究者提出了多种全自动化仿真方案，主要包括以下三类。

1.2.1.1 基于原始硬件的外设建模方案

解决外设依赖的一种方法是在底层内存映射寄存器接口上实现硬件，或通过观察真实硬件的行为来构建外设模型。然而这是一项费力的任务，扩展性不好。当前

主流的仿真器 QEMU 也仅能仿真 10 余种设备，远不能满足固件仿真工作的需求。

Gustafson 等人提出的 Pretender[12] 工具通过观察固件与物理设备之间真实发生的交互记录，使用机器学习、模式识别等方法为每个外设构建模型，再结合 QEMU 或 angr 等仿真引擎实现固件仿真。这种方法需要真实设备的支持，但在观测到足够的交互模式后，可以自动生成较为准确的外设模型。

1.2.1.2 基于程序分析的外设建模方案

基于程序分析的外设建模方案通过对固件代码进行静态或动态分析，提取固件与外设之间的交互逻辑，构建虚拟外设模型以替代真实硬件。

Feng 等人提出的 P2IM[13] 工具通过动态符号执行和模糊测试，自动识别固件中的输入点和输出点，构建与真实设备相似的虚拟模型。Cao 等人提出的 Laelaps[14] 在因访问未实现的外设而卡住时，切换到符号执行以找到正确输入，引导 QEMU 找到与实际执行最相近的路径。Zhou 等人提出的 uEmu[15] 工具利用符号执行分析固件镜像，将未知的外设寄存器表示为符号，构建推断规则并存储在知识库中。Scharnowski 等人提出的 Fuzzware[8] 使用精确的 MMIO 建模实现有效的固件模糊测试，通过确定固件逻辑如何实际使用硬件生成的值，自动生成模型以提高模糊测试输入数据的有效性。

1.2.1.3 基于系统或驱动层替换的外设仿真方案

一种有前途的方法是基于系统或驱动层替换的仿真技术，或称为高级仿真（High Level Emulation, HLE）技术。它利用固件中的公共抽象来消除为外设提供低级支持的需要。

Chen 等人提出了一个针对 Linux 内核的物联网设备固件仿真平台 Firmadyne[7]，通过修改内核和驱动来添加硬件支持，实现物联网设备的全系统仿真。该工具解决了定制化外围 I/O、非易失性内存、动态文件等仿真技术难题。Li 等人创建了 Para-rehosting[16] 工具，该工具的主要思想是利用 POSIX 接口抽象并实现了一个便携式 MCU，以将嵌入式系统源代码重新编译到商用机器的本机可执行文件中，以便利

用 x86 上许多现有的复杂软件测试工具对代码进行测试。然而，这种方案的实现需要分析师获得固件样本的源代码，而在第三方的固件分析工作中，源代码的获取往往是困难甚至完全不可行的。Clements 等人提出了 HALucinator[17] 工具，该工具利用了微控制器制造商提供的硬件抽象层（Hardware Abstraction Layer, HAL），通过对硬件抽象层函数的抽象与替换进行固件仿真，这项工作在重托管简单的裸金属固件程序时取得了很好的效果。

然而，上述基于系统或驱动层替换的仿真技术自动化程度较差，难以适应大规模安全测试的需要。具体而言，这类技术面临以下两个核心难点：一是难以自动化定位需要进行替换的驱动程序。在嵌入式程序中，识别哪些部分存在硬件依赖并需要替换，往往依赖分析人员的手动干预。手动分析嵌入式程序的复杂性使得这一过程既耗时又容易出错。二是难以自动化生成替换程序。除了识别硬件相关功能，当前的技术也缺乏自动化生成替代程序的能力。分析人员必须手动分析函数的功能，理解其与硬件和中间件的交互，并编写相应的替代代码。这个过程不仅繁琐，而且容易导致人力成本的增加。

1.2.1.4 固件仿真技术对比

表 1.1 对比了当前主流固件仿真技术的优缺点。

在这三类方案中，基于系统或驱动层替换的固件仿真方法是一种有前景的方案。它从固件系统层和驱动函数入手，较好地解决了基于程序分析和原始硬件建模的方法对复杂外设难以完全模拟或建模工作量大等痛点。HALucinator 等工具利用微控制器制造商提供的硬件抽象层（Hardware Abstraction Layer, HAL），通过函数替换的方式实现固件仿真，在重托管简单的裸金属固件程序时取得了很好的效果。

然而，现有驱动层替换方案在自动化方面仍存在明显不足：一方面，驱动函数的定位依赖分析人员的手动干预，识别固件中哪些部分存在硬件依赖并需要替换是一个耗时且容易出错的过程；另一方面，替换程序的生成也缺乏自动化支持，分析人员需要手动分析函数功能、理解其与硬件的交互，并编写相应的替代代码。为解决上述问题，需要设计一种能够自动化定位需要替换的驱动函数、并利用大模型或

表 1.1 当前主流固件仿真技术对比

方案	典型工具	优点	缺点
基于原始硬件的外设建模	QEMU, Pretender	可对外设行为进行精确模拟，提高测试效率和覆盖率	需要为每种外设专门建模，工作量大，通用性差，依赖真实设备观测
基于程序分析的外设建模	P2IM, uEmu, Laelaps, Fuzzware	可根据程序自身信息自动推断外设行为，减少人工干预，提高通用性	难以完全覆盖复杂外设的所有行为，可能存在误报或漏报
基于系统或驱动层替换	HALucinator, Firmadyne, Para-rehosting	在系统层或驱动层对函数进行抽象和替换，降低对硬件的依赖，提高测试速度和灵活性	驱动函数的定位与替换依赖人工干预，自动化程度不足，难以适应大规模安全测试

代码分析技术自动生成替代程序的方案，以提升固件仿真过程的自动化程度，实现高效的物联网设备程序仿真与动态测试。

1.2.2 嵌入式系统模糊测试技术发展现状

嵌入式系统模糊测试技术是一种针对嵌入式设备的软件安全测试方法，它可以自动或半自动地生成随机或变异的输入数据，发送给嵌入式设备的 Web 服务或其他接口，监控设备的运行状态，发现潜在的漏洞或缺陷。目前，嵌入式系统模糊测试技术主要从以下三个方向进行发展：

1.2.2.1 面向真实设备的黑盒模糊测试

直接对嵌入式设备进行模糊测试，不需要对设备进行任何修改或插桩，只需要获取设备的网络接口或串口等信息，然后使用模糊测试工具生成并发送测试用例，监控设备的响应和异常。Chen 等人开发的 IoTFuzzer[18] 工具，就利用了应用程序与物联网设备的交互信息，通过网络接口向嵌入式设备发送模糊测试数据，进行模糊测试工作。这种方法的优点是简单易用，不依赖于设备的内部结构或代码，适用于黑盒测试场景。缺点是测试效率较低，覆盖率较差，难以发现深层次的漏洞。

1.2.2.2 基于固件仿真的灰盒模糊测试

基于固件仿真的模糊测试，通过对嵌入式设备的固件进行逆向工程，提取出可执行的二进制代码，然后使用仿真器在虚拟环境中运行固件，并对其进行模糊测试。基于该方案的仿真工具有 Fuzzware[8]、uEmu[15]、HALucinator[17] 等。这种方法的优点是可以对固件进行插桩或修改，获取更多的运行信息，提高测试效率和覆盖率，发现更多的漏洞。缺点是需要对固件进行复杂的逆向分析，可能遇到加密或保护机制，同时仿真器可能存在不完全或不准确的问题，导致测试结果与真实设备不一致。

1.2.2.3 基于静态分析辅助的模糊测试方案

固件分析是一种辅助模糊测试的技术，它通过对嵌入式设备的固件进行静态或动态分析，获取固件的结构、功能、协议等信息，然后根据这些信息生成更有效的测试用例，提高模糊测试的针对性和有效性。例如，Firmalice[19] 便是利用固件分析技术进行模糊测试的工具。其有三个主要组成部分：静态程序分析模块、符号执行和认证旁路检查模块。这种技术的优点是可以利用固件中已有的数据样本或语法规则，生成更符合设备输入要求的测试用例，触发更多的程序路径和异常。缺点是需要对固件进行深入的分析，可能需要专业的知识和工具。

1.2.3 大模型程序分析与代码生成技术研究现状

大语言模型（Large Language Model, LLM）在代码理解和生成方面展现出了强大的能力。以 GPT-4[20]、Code Llama[21]、StarCoder[22] 等为代表的代码大模型能够理解复杂的代码逻辑，并生成高质量的代码。近年来，研究者开始探索将 LLM 应用于程序分析和代码生成领域，涌现出多种增强策略以提升模型在复杂编程任务中的表现。

1.2.3.1 面向特定任务的代码生成方法

早期 LLM 通过零样本提示能够生成基础代码，但在处理复杂编程任务时准确性和可靠性仍存在不足。为克服这些局限，研究者发展了以下增强策略：

指令微调（Instruction Tuning）：通过对预训练 LLM 在特定领域数据集上进行

精细调整，显著提升模型对特定编程任务的理解和执行能力[23]。DolphCoder 模型通过多样化的指令和自评估机制，结合代码评估目标有效提升了代码生成能力[24]。CodeLSI 框架则通过低秩优化与领域特定指令微调相结合，提高了代码生成的领域特异性和成本效益[25]。

测试驱动的迭代生成：利用测试用例作为反馈信号指导模型逐步修正生成的代码，直至通过所有测试。这种方法通过隐式执行监督信号，结合搜索、重排序或迭代修正等机制动态优化模型输出[26]。RLCF（Reinforcement Learning with Compiler Feedback）方法通过编译器反馈对 LLM 进行额外训练，解决生成代码中可能存在的语言级不变量违规问题[27]。CodeDPO 框架则将偏好学习整合到代码生成过程中，优化模型优先选择正确且高效的解决方案[28]。

多步推理方法：思维链（Chain-of-Thought, CoT）和程序思维（Program-of-Thought, PoT）等方法通过将复杂问题分解为多个可管理的子步骤，提升模型解决复杂问题的能力[29]。MoTCoder 引入模块化思维指令框架来应对更具挑战性的编程任务[30]。程序思维方法则将中间推理步骤编译为可执行代码片段，实现了”推理即执行”，大幅提升了数值计算和 API 调用类任务的准确性。

工具增强的大模型：通过集成编译器、调试器、静态分析工具等外部工具，扩展模型的实际应用能力。这种范式将 LLM 定位为”AI 协调者”，通过标准化工具调用协议调度各种工具，形成感知-决策-执行的闭环[31]。MCCoder 系统专注于生成运动控制代码，并结合严格的验证机制以确保代码安全性[32]。RGD 代理调试器利用多 LLM 通过细化和生成指导来自动修复代码中的错误[33]。

1.2.3.2 系统级软件代码生成的挑战与前沿

在系统级软件应用层面，特别是针对物联网设备驱动程序、操作系统内核和嵌入式固件等复杂系统，LLM 的代码生成面临独特挑战。IoT 设备驱动程序开发具有强硬件耦合性、并发中断敏感性、底层资源约束性以及隐式的 DMA/寄存器交互等特征，对 LLM 的代码生成能力提出了更高要求[34]。

当前前沿研究强调构建”分析—生成—验证”的闭环系统以应对这些挑战：

静态分析集成：静态分析工具如 CIL、Smatch 和 Frama-C 用于提取接口契约与控制流约束，对代码进行结构化表示[35]。这些工具能够识别不安全的功能、缓冲区溢出和竞态条件等安全漏洞，为 LLM 生成提供结构化上下文。

动态符号执行验证：动态符号执行引擎如 QEMU 和 KLEE 能够生成高覆盖率的路径约束，并反向验证生成代码的内存安全与中断语义正确性[36]。通过结合编译器诊断、安全扫描和符号执行，LLM 可以迭代细化其输出。

端到端流水线：研究者正在构建端到端的" 分析 → 生成 → 符号验证 → 反馈精炼" 流水线[37]。形式化规约被用作生成约束，而验证失败轨迹则被结构化为自然语言反馈以驱动下一轮生成。Agents4PLC 框架通过基于 LLM 的代理自动化了可编程逻辑控制器（PLC）代码生成与验证，提高了工业控制系统的操作效率和安全性。

这些研究为本文利用 LLM 进行驱动函数分析和替换提供了理论基础，特别是在构建自动化分析-生成-验证闭环方面具有重要的参考价值。

1.2.4 现有研究的局限性分析

综合上述研究现状，现有固件仿真技术主要存在以下局限性：

（1）外设建模依赖人工，难以处理复杂外设行为

现有仿真方法大多依赖人工编写外设模型或基于有限规则的自动推断。硬件在环方案需要为每种外设手动配置物理设备连接；基于原始硬件的外设建模方案需要人工分析数据手册并编写仿真代码；基于程序分析的方案虽然能自动推断部分外设行为，但推断规则有限，难以覆盖复杂外设的状态机逻辑。例如，网络控制器、USB 设备等复杂外设具有多通道、DMA 传输、中断嵌套等特性，现有自动推断方法往往只能处理简单的 MMIO 寄存器读写，无法准确模拟这些复杂交互行为。

（2）缺乏对驱动函数语义的深入理解，仿真结果可能不准确

现有驱动层替换方案（如 HALucinator）主要通过函数 Hook 的方式进行替换，即当固件调用某个 HAL 函数时，返回预设的模拟值或执行简化的模拟逻辑。然而，这种方法缺乏对驱动函数内部语义的深入理解：函数之间的调用关系、参数的数据流传递、外设状态的变化等关键信息往往被忽略。这导致在某些场景下，仿真结果与真实硬件行为存在偏差，可能遗漏重要的程序路径或引入虚假的漏洞。

（3）缺乏自动化反馈机制，难以迭代优化仿真质量

当前大多数仿真工具采用”一次性配置”的模式：分析人员手动配置仿真环境后运行测试，如果发现问题则需要人工介入调试和修复。这种模式缺乏自动化的反馈闭环机制——当仿真过程中出现异常（如程序卡死、断言失败、外设访问超时）时，系统无法自动分析原因并调整仿真策略。这导致仿真质量的提升高度依赖分析人员的经验，难以适应大规模、多样化的固件测试需求。

1.3 研究内容与主要贡献

为解决上述问题，本文提出一种基于驱动程序替换的物联网设备固件仿真方法。该方法通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型分析驱动函数语义并自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量。本文主要的创新点和贡献有三个：

（1）本研究设计并实现了一个面向嵌入式固件的硬件依赖代码自动识别模块。该模块通过 MMIO 模式识别、数据流追踪与递归归纳等技术，从固件二进制代码中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界，构建可查询的驱动知识库。与现有依赖人工标注或有限规则推断的方法相比，该模块能够自动处理多种嵌入式平台（STM32、NXP 等）和操作系统（FreeRTOS、RT-Thread、裸机）的固件，显著降低了驱动函数定位的人工成本。

（2）本研究设计并实现了一个基于大语言模型的驱动函数分析与替换模块。该模块利用大语言模型的代码理解能力，对识别出的驱动函数进行语义分析，理解其与硬件的交互逻辑，并自动生成可在仿真环境中运行的替代实现。通过设计结构化的提示词模板和函数分类策略，本模块能够处理不同类型的驱动函数（如初始化、数据传输、中断处理等），并通过编译修复循环保障生成代码的可编译性。与现有基于预设返回值的简单 Hook 方法相比，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为。

（3）本研究引入了动态测试反馈闭环机制。当固件在仿真环境中运行时，该机制能够捕获运行时异常（如程序卡死、外设访问超时、断言失败等），分析异常原因，

并将分析结果反馈给驱动替换模块以迭代优化替代代码。这种”生成-测试-反馈-修正”的闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖。实验结果表明，该反馈机制能够在有限的迭代轮次内显著提高固件在仿真环境中的稳定性和运行覆盖率。

1.4 论文组织结构

本文共分为五章，组织结构如下：

第一章：绪论。介绍研究背景、意义、国内外研究现状以及本文的主要贡献。

第二章：背景知识与相关技术。介绍物联网设备硬件基础、嵌入式操作系统与驱动框架、大模型代码分析技术以及固件仿真与模糊测试基础。

第三章：系统设计与实现。详细描述系统的整体架构设计、各核心模块的设计与实现。

第四章：实验设计与结果分析。介绍实验环境、测试案例、实验结果及分析。

第五章：总结与展望。总结本文工作，讨论局限性，并展望未来研究方向。

2 背景知识与相关技术

本章介绍了物联网设备固件仿真研究涉及的基础知识，包括物联网设备硬件基础、物联网操作系统与驱动框架、大模型代码分析技术以及固件仿真与模糊测试技术，为后续章节的论述奠定基础。

2.1 物联网设备硬件基础

物联网设备通常采用嵌入式微控制器（Microcontroller Unit, MCU）作为核心处理器。常见的 MCU 架构包括 ARM Cortex-M 系列、RISC-V 等。这些处理器具有低功耗、低成本、实时性强等特点，广泛应用于物联网领域。本节重点介绍 ARM Cortex-M 架构的特性以及内存映射 I/O 机制。

2.1.1 ARM Cortex-M 架构特性

ARM Cortex-M 是 ARM 架构处理器核心中低端系列的统称，由 ARM 公司所授权。这组核心的特点为低成本以及高能源效率的微处理器优化设计，已有上千万个消费性设备中有此系列的微处理器。ARM Cortex-M 系列微处理器采用了 Thumb-2 指令集，提供了 32 位的性能和 16 位的代码密度。同时该系列提供了一些特殊的功能，如嵌套向量中断控制器（Nested Vectored Interrupt Controller, NVIC）、系统计时器（System Tick Timer, Systick）、位带存储器（Bit-Band Memory）、存储器保护单元（Memory Protect Unit, MPU）等，可以提高系统的可靠性和实时性。

2.1.1.1 嵌套向量中断控制器（NVIC）

NVIC 是 ARM Cortex-M 架构的一个核心组件，负责处理外部和内部的中断请求。嵌入式操作系统可以通过配置 NVIC 的优先级寄存器和使能寄存器，来设置不同中断源的优先级和使能状态，从而实现中断的响应和屏蔽。同时，操作系统可以通过读取 NVIC 的挂起寄存器和活动寄存器，来判断当前是否有中断发生，以及哪些中断正在执行。这些信息对于任务调度和中断嵌套是非常重要的。

2.1.1.2 系统定时器（Systick）

Systick 是 ARM Cortex-M 架构的另一个核心组件，负责提供一个可编程的周期性中断。嵌入式操作系统通过配置 Systick 的重装载寄存器和控制寄存器，来设置 Systick 的时钟源、计数模式和中断使能。通常，操作系统会将 Systick 设置为产生一个固定频率的中断，比如 1ms 或 10ms，作为操作系统的心跳。这样，操作系统就可以利用 Systick 中断来实现时间管理和任务切换等功能。

2.1.1.3 栈模式切换

主栈与进程栈是 ARM Cortex-M 架构支持的两种栈模式。主栈是默认的栈模式，用于执行异常处理程序（包括复位、NMI、硬件错误和 SVC 等）。进程栈是可选的栈模式，用于执行线程模式下的用户程序。操作系统可以通过配置控制寄存器的 SPSEL 位，来选择当前使用哪种栈模式。栈模式的切换是实现多任务并发和内存保护的基础。

2.1.2 内存映射 I/O

内存映射 I/O（Memory-Mapped I/O, MMIO）是嵌入式系统中外设与处理器通信的主要方式。与端口映射 I/O（Port-Mapped I/O, PMIO）不同，MMIO 不需要专用的 I/O 指令，而是将外设寄存器映射到处理器的统一内存地址空间中。处理器通过普通的内存读写指令（如 LDR/STR）访问特定内存地址，即可完成对外设的控制和数据传输。这种设计简化了编程模型，使得外设访问与内存访问具有相同的语法形式。

2.1.2.1 MMIO 地址空间布局

ARM Cortex-M 架构采用统一的 32 位地址空间，理论上可寻址 4GB 内存。虽然具体的地址映射可能因芯片厂商和型号而有所不同，但 ARM 定义了推荐的内存映射框架，将地址空间按功能划分为若干区域。

代码区位于地址空间的低端，用于存储程序指令和常量数据。该区域支持指令取指操作，通常映射到 Flash 存储器或其他非易失性存储器。SRAM 区紧随其后，用于存储变量、堆栈和动态分配的内存。部分 Cortex-M 处理器支持位带（Bit-Band）操

作，允许对单个位进行原子访问，这在嵌入式编程中非常有用。

外设区是 MMIO 的核心区域，所有片上外设的寄存器都被映射到此区域。不同厂商的外设被分配到不同的地址段，例如某厂商的 GPIO 可能位于某一地址范围，而 USART 位于另一地址范围。这种映射使得处理器可以通过标准的内存访问指令来控制外设。

系统区位于地址空间的高端，包含处理器核心相关的调试接口和系统控制寄存器。嵌套向量中断控制器（NVIC）、系统定时器（SysTick）、内存保护单元（MPU）等系统级组件的控制寄存器都位于此区域。此外，系统区还包含厂商特定的系统配置寄存器。

理解 MMIO 地址空间布局对于固件仿真至关重要。仿真器需要正确处理对不同区域的访问：对于代码区和 SRAM 区，可以采用标准的内存模拟；对于外设区和系统区，则需要根据具体外设的行为实现专门的访问处理逻辑。

2.1.2.2 外设寄存器分类

外设寄存器按照功能可分为控制寄存器、状态寄存器和数据寄存器三类。控制寄存器（Control Register）用于配置外设的工作模式和参数，通常是可读写的，固件通过写入特定值来设置外设功能。例如，USART 的控制寄存器 CR1 可以设置波特率、数据位长度、校验位等参数。控制寄存器的写入操作通常不会立即产生硬件响应，而是配置外设在后续操作中的行为。

状态寄存器（Status Register）反映外设的当前工作状态，通常是只读的，固件通过读取状态寄存器来判断外设是否就绪、是否发生错误等。例如，USART 的状态寄存器 SR 包含 TXE（发送缓冲区空）、RXNE（接收缓冲区非空）等标志位。状态寄存器的值由硬件根据外设实际状态自动更新，这是 MMIO 与普通内存访问的关键区别。

数据寄存器（Data Register）用于数据传输，通常是可读写的，写入操作将数据发送到外设，读取操作从外设接收数据。例如，USART 的数据寄存器 DR 用于发送和接收串口数据。对数据寄存器的访问通常会触发外设执行相应操作（如开始数据传输），并可能改变状态寄存器的值。

2.1.2.3 MMIO 结构体定义

嵌入式厂商通过 C 语言结构体定义外设寄存器的内存布局。典型的 MMIO 结构体定义如下：

```
typedef struct
{
  __IO uint32_t CR1;    // Control register 1
  __IO uint32_t CR2;    // Control register 2
  __IO uint32_t SR;     // Status register
  __IO uint32_t DR;     // Data register
} USART_TypeDef;

#define USART1 ((USART_TypeDef *)0x40011000)
#define USART2 ((USART_TypeDef *)0x40004400)
```

其中 `__IO` 宏定义了寄存器的读写属性（volatile 关键字确保编译器不优化掉对寄存器的访问）。通过将固定地址强制转换为结构体指针，固件代码可以使用结构体成员访问语法来操作寄存器，如 `USART1->CR1 = 0x2000`。

2.1.2.4 MMIO 访问模式分析

固件代码中定位和访问 MMIO 结构体的典型模式包括直接地址强转、间接初始化和常量指针参数三种。直接地址强转模式将固定地址直接强制转换为外设结构体指针，这是最直接的模式，常见于裸机程序或简单固件中，例如通过 `USART_TypeDef *usart = (USART_TypeDef *)0x40011000` 直接获取 USART1 的指针，然后通过 `usart->CR1 = 0x2000` 来使能 USART。

间接初始化模式通过配置表或句柄将外设基地址赋值给结构体成员，这是 HAL 库的标准模式，支持多实例管理。例如在 HAL 库中，首先定义 `UART_HandleTypeDef huart1` 句柄，然后通过 `huart1.Instance = USART1` 将 Instance 成员指向外设基地址，最后调用 `HAL_UART_Init(&huart1)` 完成初始化。这种模式的优势在于支持多个同类外设实例，通过句柄统一管理。

常量指针参数模式中，函数以指针参数形式接收外设基址，实现代码复用。例如 `void USART_SendByte(USART_TypeDef *usart, uint8_t data)` 函数接收 USART 指针和数据，通过 `usart->DR = data` 写入数据寄存器，然后通过 `while (!(usart->SR & USART_SR_TXE))` 轮询状态寄存器等待发送完成。这种模式将 MMIO 访问封装在函数内部，提高了代码的可维护性和可移植性。

这些访问模式是静态分析识别硬件依赖代码的重要依据。通过分析代码中对 MMIO 结构体指针的来源和使用方式，可以准确定位固件中的硬件访问点。

2.1.2.5 MMIO 在固件仿真中的挑战

MMIO 机制给固件仿真带来了独特的技术挑战，这些挑战源于硬件外设的复杂性和 MMIO 访问模式的特殊性。

（1）寄存器语义的多样性与不确定性

不同类型的外设具有完全不同的寄存器布局和行为语义。例如，UART 外设的数据寄存器用于串口通信，而 GPIO 外设的数据寄存器用于控制引脚电平。即使是同一类型的外设，不同厂商的实现也可能存在差异，寄存器数量、位域定义、操作时序都可能不同。这种多样性使得难以构建通用的外设仿真模型。更复杂的是，许多嵌入式固件使用厂商自定义的外设，这些外设的文档可能不完整甚至完全缺失，导致仿真器难以准确理解寄存器的语义。

（2）状态寄存器的动态响应问题

状态寄存器的值不是固定的，而是根据外设的内部状态动态变化的。例如，UART 的状态寄存器中的接收缓冲区非空（RXNE）标志位，只有当外设实际接收到数据时才会置位。在仿真环境中，如果没有真实的数据输入，这个标志位应该如何响应？简单的静态返回值无法准确模拟这种行为，而完全模拟外设的状态机又需要深入理解外设的工作原理。这种状态依赖性使得固件可能因为等待某个永远不会出现的状态而无限循环。

（3）寄存器访问的副作用

对 MMIO 寄存器的某些访问会产生副作用，这是 MMIO 与普通内存访问的关键区别。例如，读取 UART 的数据寄存器不仅返回接收到的字节，还会自动清除接收缓冲区非空标志；写入中断清除寄存器的某些位会清除对应的中断挂起状态。如果仿真器不能正确模拟这些副作用，固件的执行逻辑就会出现错误。识别和理解这些副作用需要仔细分析外设文档，或者通过动态观察真实硬件的行为来推断。

（4）多寄存器操作的原子性与时序约束

许多外设操作涉及多个寄存器的协调访问。例如，配置一个外设可能需要按特定顺序写入多个控制寄存器，如果顺序错误可能导致配置失败或外设状态异常。某些外设对时序有严格要求，如在设置使能位后必须等待一定周期才能访问数据寄存器。这些时序约束在简单的寄存器读写模拟中难以表达，但在真实硬件上是必须遵守的。仿真器需要在保持足够精度的同时，又不能过度限制以避免影响性能。

这些挑战使得传统的内存模拟方法无法直接应用于 MMIO 区域，需要针对不同外设实现专门的仿真逻辑。正是这些复杂性，驱动了基于驱动层替换的仿真方法——通过识别和替换访问 MMIO 的驱动函数，可以在函数调用层面进行抽象，避免直接模拟底层硬件的复杂行为。

这些挑战使得传统的内存模拟方法无法直接应用于 MMIO 区域，需要针对不同外设实现专门的仿真逻辑。这正是驱动层替换方法的核心动机——通过识别和替换访问 MMIO 的驱动函数，避免直接模拟复杂的硬件行为。

2.2 物联网操作系统与驱动

2.2.1 实时操作系统概述

物联网设备常采用实时操作系统（Real-Time Operating System, RTOS）来管理任务调度和资源分配。RTOS 能够及时响应外部事件或数据，并在规定的时间内完成任务。RTOS 的工作原理是通过一个实时任务调度器来管理系统资源，按照优先级或者时间片来分配 CPU 时间给不同的任务，保证重要或者紧急的任务优先执行。

FreeRTOS[3] 是 Amazon 开源的实时操作系统，专为微控制器和小型微处理器设计。它采用抢占式调度策略，支持任务优先级和队列通信机制。FreeRTOS 的内核非常精简，核心代码仅数千行，最小可运行在不到 10KB 的 RAM 上，这使得它能够运行在资源极度受限的嵌入式设备上。FreeRTOS 提供了丰富的 API 接口，包括任务管理、队列、信号量、互斥量、软件定时器等功能模块，方便开发者构建复杂的应用程序。由于其开源免费、许可证宽松的特点，FreeRTOS 已成为物联网领域使用最广泛的 RTOS 之一。

Zephyr[4] 是 Linux 基金会管理的开源实时操作系统，支持多种处理器架构（包

括 ARM、RISC-V、x86 等）和数百种开发板。与 FreeRTOS 不同，Zephyr 提供了更完整的操作系统功能，包括文件系统、网络协议栈、设备驱动模型、安全特性（如内存保护、安全启动）等。Zephyr 采用模块化设计，开发者可以根据需要选择包含或排除特定功能，从而在功能丰富性和资源占用之间取得平衡。Zephyr 的设备驱动框架定义了统一的驱动接口规范，这使得不同厂商的硬件可以通过相同的 API 访问，降低了应用开发的复杂度。

RT-Thread[38] 是中国开源社区主导开发的实时操作系统，定位于" 小而美" 的物联网操作系统。RT-Thread 采用微内核架构，核心组件包括线程调度、线程间通信、内存管理、设备驱动等，并提供了丰富的组件生态，如文件系统、网络协议栈、图形界面等。RT-Thread 的一个显著特点是其设备驱动框架（Device Driver Framework, DDF），它为不同类型的外设（如串口、SPI、I2C、GPIO 等）定义了统一的操作接口，驱动开发者只需实现这些接口即可完成驱动移植。RT-Thread 还提供了软件包管理系统，开发者可以方便地集成第三方库和组件。在国内物联网领域，RT-Thread 得到了广泛的应用和活跃的社区支持。

2.2.2 硬件抽象层

硬件抽象层（Hardware Abstraction Layer, HAL）是嵌入式软件架构中的关键组成部分，它位于硬件驱动层和上层应用之间，为应用程序提供统一的硬件访问接口。HAL 层的核心目标是屏蔽底层硬件的差异性，使得上层应用代码可以在不同的硬件平台之间移植，而无需修改业务逻辑。

在传统的嵌入式开发中，应用程序需要直接操作寄存器来完成外设控制，这种方式不仅代码可读性差，而且当更换硬件平台时需要大量修改。HAL 层通过定义统一的接口规范，将硬件操作封装为函数调用，显著提高了代码的可维护性和可移植性。例如，不同厂商的 UART 外设寄存器布局可能完全不同，但通过 HAL 层，应用程序可以使用统一的 `HAL_UART_Transmit()` 函数来发送数据，而无需关心底层寄存器的具体实现。

HAL 层的接口设计通常遵循一定的命名和功能规范。以 STM32 的 HAL 库为例，每个外设类型都有一组标准化的接口函数：初始化函数（如 `HAL_UART_Init`）负责配

置外设参数和使能时钟；数据传输函数（如 HAL_UART_Transmit、HAL_UART_Receive）封装了数据的发送和接收逻辑；状态查询函数（如 HAL_UART_GetState）用于获取外设的当前工作状态；中断管理函数（如 HAL_UART_Receive_IT）用于配置中断模式和注册回调函数。这种标准化的接口设计使得开发者可以快速熟悉不同外设的使用方法。

HAL 层的抽象特性为固件仿真提供了重要基础。由于 HAL 函数封装了具体的硬件操作细节，仿真器可以通过替换 HAL 层函数来实现硬件行为的模拟，而无需深入理解底层寄存器的语义。这种函数级的替换相比寄存器级的模拟更加高效，也更容易实现。在实际的固件仿真系统中，通过识别和替换 HAL 层的硬件相关函数，可以有效解耦固件逻辑与硬件依赖，使固件能够在没有真实硬件的环境中正常运行。

2.2.3 驱动函数分类

驱动函数是嵌入式固件中与硬件交互的核心组件。从功能角度，驱动函数可以划分为初始化与配置函数、数据传输函数、中断与回调函数、状态查询与控制函数四类。

初始化与配置函数负责外设的初始化和参数配置，是外设正常工作的前提。初始化函数通常执行以下操作：使能外设时钟、配置 GPIO 引脚复用、设置通信参数（如波特率、数据位、校验位等）、使能外设功能模块。例如，HAL_UART_Init 函数会根据传入的配置结构体设置 UART 外设的各项工作参数，并使能 UART 模块。初始化函数的执行结果直接影响后续数据传输的正确性，因此在仿真环境中需要确保初始化函数能够正确设置仿真模型的内部状态。

数据传输函数是驱动程序的核心，负责在处理器和外设之间传递数据。根据数据流向，可以分为发送函数和接收函数。发送函数（如 HAL_UART_Transmit）将数据写入外设的数据寄存器，触发硬件发送；接收函数（如 HAL_UART_Receive）从数据寄存器读取外设接收到的数据。数据传输函数通常涉及状态查询（等待发送缓冲区空或接收缓冲区非空）、超时处理、错误检测等逻辑。在仿真环境中，这类函数需要与仿真模型的数据缓冲区进行交互，并正确模拟状态变化。

中断与回调函数处理嵌入式系统中的异步事件。中断管理函数（如 HAL_UART_Receive_IT）

用于配置外设的中断使能和回调函数指针。当外设产生中断事件（如接收完成、发送完成、错误等）时，中断服务程序会调用预先注册的回调函数，通知应用程序处理相应事件。回调函数由应用程序实现，用于响应特定的硬件事件。在仿真环境中，需要模拟中断的产生时机和回调函数的调用逻辑，这对保证固件执行流程的正确性至关重要。

状态查询与控制函数为应用程序提供监控和控制外设的手段。状态查询函数（如 HAL_UART_GetState）用于获取外设的当前工作状态，包括是否忙、是否就绪、是否有错误等。控制函数（如 HAL_UART_DeInit）用于复位外设或改变其工作模式。在仿真环境中，状态查询函数需要返回与仿真模型内部状态一致的结果，控制函数需要正确更新仿真模型的状态。以上分类涵盖了驱动函数的主要功能类型，在实际的固件仿真系统中，针对不同类型的驱动函数需要采用不同的替换策略：初始化函数主要用于设置状态，可以简化实现；数据传输函数需要与仿真模型交互，实现较为复杂；中断相关函数需要正确模拟异步事件；状态查询函数需要返回一致的状态信息。理解驱动函数的功能分类，有助于设计更加精确和高效的仿真替换策略。

2.3 大模型代码分析智能体框架

大语言模型（Large Language Model, LLM）在近年来取得了突破性进展，其强大的代码理解和生成能力为自动化程序分析提供了新的技术途径。本节简要介绍大语言模型的发展历程，重点阐述通用大模型的代码能力以及基于 LLM 的智能体开发框架，为后续章节的驱动函数分析和替换工作奠定技术背景。

2.3.1 大语言模型发展概述

大语言模型的发展始于 2017 年 Transformer 架构的提出。Transformer 完全基于自注意力机制，摒弃了传统的循环神经网络结构，能够高效地建模长距离依赖关系。这一架构创新为后续的大规模预训练语言模型奠定了基础。

以 GPT 系列为代表的仅解码器（Decoder-only）架构通过大规模无监督预训练，展现出强大的语言理解和生成能力。随着模型规模从数亿参数扩展到数千亿参数，这些模型逐渐涌现出少样本学习（Few-shot Learning）、思维链推理（Chain-of-Thought

Reasoning）等高级能力。特别是 GPT-4、Claude-3 等新一代模型，不仅在自然语言处理任务上表现优异，在代码生成、程序理解、调试辅助等编程相关任务上也展现出接近甚至超越人类专家的能力。

近年来，开源大模型生态也快速发展。LLaMA、Mistral、Qwen 等开源模型在多个基准测试中取得了与闭源模型相当的性能，为学术研究和工业应用提供了更多选择。这些开源模型的出现，使得在本地部署和定制化训练大语言模型成为可能，降低了技术门槛。

2.3.2 通用大模型的代码能力

大语言模型的代码生成与理解能力，近年来取得了突破性进展，已成为衡量模型性能的核心指标之一。从最初的简单代码补全，到如今能够完成复杂项目开发、跨文件代码重构、技术债务分析等高级任务，大模型的编程能力正以前所未有的速度迭代升级。代码数据因其逻辑严谨、结构清晰的特点，本身也是提升模型推理能力的重要训练素材——一个模型如果擅长写代码，往往在数学、逻辑推理等任务上也表现优异。正因如此，主流大模型厂商纷纷将代码能力作为重点突破方向，推动模型从”能写简单函数”向”理解完整项目”演进。本研究主要使用以下通用大语言模型。

DeepSeek-V3.2 是深度求索公司发布的新一代大语言模型，在代码和推理能力上达到了全球领先水平。该模型采用 DSA（DeepSeek Sparse Attention）稀疏注意力机制，在保持无损长文本性能的同时大幅降低了模型部署成本。DeepSeek-V3.2 的核心创新在于将思考推理融入工具使用，这是首个支持思考模式下工具调用的模型。通过大规模 Agent 训练数据合成方法，模型在 1800 多个环境、85000 多个复杂指令上进行了强化学习训练，显著提高了泛化能力。在公开的推理类基准测试中，DeepSeek-V3.2 达到了 GPT-5 的水平。其长思考增强版 DeepSeek-V3.2-Speciale 更是斩获了 IMO 2025（国际数学奥林匹克）、CMO 2025（中国数学奥林匹克）、ICPC World Finals 2025（国际大学生程序设计竞赛全球总决赛）及 IOI 2025（国际信息学奥林匹克）金牌，其中 ICPC 与 IOI 成绩分别达到了人类选手第二名与第十名的水平。

GLM-5 是智谱 AI 发布的新一代旗舰基础模型，专为 Agentic Engineering 设计，能够在复杂系统工程和长距离 Agent 任务中提供可靠的生产力。GLM-5 将参数规模

从 355B（32B 激活）扩展到 744B（40B 激活），预训练数据从 23T 升级到 28.5T，更大规模的预训练计算显著提升了模型的通用智能。模型采用异步强化学习框架 Slime，支持更大模型规模和更复杂强化学习任务，使模型能够从长距离交互中持续学习。在软件工程任务上，GLM-5 实现了与 Claude Opus 4.5 的性能对齐，在 SWE-bench Verified 上取得了 77.8 的开源模型最高分，在 Terminal Bench 2.0 上取得了 56.2 的领先成绩。在智能体能力方面，GLM-5 在 BrowseComp（网页级检索与信息合成）、MCP-Atlas（工具调用与多步任务执行）、τ^2-Bench（复杂多工具规划与编排）等多个权威基准测试上均取得了开源模型第一的成绩。

MiniMax-M2.5 是 MiniMax 公司发布的新一代大语言模型，经过数十万个真实复杂环境中的大规模强化学习训练，在编程、工具调用和生产力场景中达到了行业顶尖水平。在 SWE-Bench Verified 上，M2.5 取得了 80.2% 的通过率；在 Multi-SWE-Bench 上达到了 51.3%，在多语言任务上排名第一；在 BrowseComp 上取得了 76.3% 的成绩。M2.5 具备" 像架构师一样思考和构建" 的能力，演化出了原生 Spec 行为：在动手写代码前，以架构师视角主动拆解功能、结构和 UI 设计，实现完整的前期规划。模型在超过 10 种编程语言（包括 Go、C、C++、TypeScript、Rust、Kotlin、Python、Java、JavaScript、PHP、Lua、Dart、Ruby）和数十万个真实环境中进行了训练，能够胜任从 0-1 系统设计、从 1-10 系统开发、从 10-90 功能迭代、从 90-100 完备代码审查与系统测试的全流程。M2.5 通过 Forge 原生 Agent RL 框架实现了大规模强化学习，在包括大量真实任务的数十万个 Agent 环境上进行训练，验证了模型能力随算力和任务数的 scaling 取得近线性提升。此外，M2.5 提供了极具竞争力的成本优势：在每秒输出 100 token 的情况下，连续工作一小时只需 1 美金，使得在经济上几乎无限制地构建和运营 Agent 成为可能。

通用大模型在代码相关任务上的成功，主要归因于大规模预训练、强化学习对齐和智能体架构三个方面。大规模预训练使模型在包含大量代码的数据集上学习到丰富的编程知识和代码模式；强化学习对齐通过数万到数十万个真实环境的训练，显著提升了模型在复杂任务中的决策成熟度和泛化能力；智能体架构则通过长距离规划、工具调用和多步执行，使模型能够完成从需求理解到交付完整产物的全流程

任务。这些进展使得通用大模型从单纯的代码生成工具，演化为能够理解完整项目、自主规划执行、持续迭代优化的智能助手。

2.3.3 LLM 智能体开发框架

为了更好地利用大语言模型构建复杂的应用系统，研究人员开发了多种智能体（Agent）框架。这些框架提供了结构化的方式来组织 LLM 的调用、管理状态、集成外部工具，使开发者能够构建可靠的、可扩展的 LLM 应用。

2.3.3.1 LangChain 框架

LangChain 是目前最流行的 LLM 应用开发框架之一，提供了丰富的组件和抽象，帮助开发者快速构建 LLM 驱动的应用。LangChain 的核心概念包括模型、提示模板、链、代理、工具和记忆。模型组件封装了与不同 LLM 提供商的交互接口，支持 OpenAI、Anthropic、本地模型等多种后端，开发者可以通过统一的接口调用不同的模型，便于切换和比较。提示模板组件允许开发者定义带变量的提示词模板，在运行时动态填充变量值，使提示词的构造更加灵活和可复用。

链是将多个组件按顺序组合的机制，例如一个代码分析链可能包含读取代码文件、构造提示词、调用 LLM、解析结果、存储分析报告等步骤。通过链的组合，可以构建复杂的处理流程。代理是更高级的抽象，它允许 LLM 自主决定下一步行动。代理根据用户的输入，选择合适的工具（如搜索引擎、代码执行器、数据库查询），执行操作，并根据结果决定后续行动，直到完成任务。LangChain 内置了多种代理类型，如 ReAct 代理、OpenAI Functions 代理等。工具是代理可以调用的外部功能，LangChain 提供了大量内置工具（如网络搜索、Python 代码执行、文件操作等），开发者也可以自定义工具。记忆组件允许应用在多次调用之间保持状态，例如在多轮对话中存储历史对话，使 LLM 能够理解上下文。LangChain 提供了多种记忆实现，如缓冲记忆、摘要记忆、向量存储记忆等。

2.3.3.2 LangGraph 状态图

LangGraph 是 LangChain 生态中用于构建有状态、可循环的智能体工作流的库。与 LangChain 的链式结构不同，LangGraph 基于状态图（State Graph）的概念，支持更复杂的控制流。LangGraph 的核心是状态（State）对象，它在工作流的各个节点之间传递和更新。状态可以包含任意类型的数据，如对话历史、中间结果、用户输入等。每个节点可以读取和修改状态，实现信息的流动和共享。

节点（Node）是工作流中的处理单元，接收状态作为输入，返回状态的更新。边（Edge）定义了节点之间的转换关系。LangGraph 支持条件边（Conditional Edge），根据状态的值动态选择下一个节点，实现分支逻辑。与简单的链式结构不同，LangGraph 支持循环，允许工作流在满足特定条件前重复执行某些节点。这对于迭代优化、多轮交互等场景非常有用。例如，在代码生成任务中，可以设计一个循环：生成代码、编译验证、如果失败则修复、重新生成，直到代码可编译。LangGraph 还支持检查点（Checkpoint）机制，可以将工作流的状态持久化存储，使得工作流可以暂停和恢复，支持长时间运行的任务。在代码分析场景中，如果 LLM 调用超时或失败，可以从上一个检查点恢复，而不需要重新开始。

2.3.4 代码分析工作流范式

基于 LLM 智能体框架，可以构建多种代码分析工作流。ReAct（Reasoning + Acting）是一种结合推理和行动的工作流范式。在代码分析任务中，LLM 首先进行推理（如”我需要理解这个函数的功能”），然后采取行动（如”调用静态分析工具提取函数的调用关系”），观察结果，再进行下一步推理。这种范式使 LLM 能够动态地收集信息，做出决策，逐步完成复杂的分析任务。

Plan-and-Execute 范式将任务分解为规划和执行两个阶段。在规划阶段，LLM 分析任务要求，生成一个执行计划（如识别硬件相关代码、分析驱动函数语义、生成替代代码、验证编译）。在执行阶段，按照计划依次执行各个步骤。这种范式的优点是结构清晰，易于调试和监控。

Multi-Agent 范式使用多个专门的智能体协作完成任务。例如，在驱动函数替换

任务中，可以设计分析智能体（负责识别驱动函数，分析其语义和依赖关系）、生成智能体（负责生成替代代码，考虑仿真环境的特点）、验证智能体（负责编译验证，检查代码的正确性）、协调智能体（负责协调各智能体的工作，处理冲突）。各智能体通过消息传递或共享状态进行协作，共同完成复杂的代码分析和生成任务。

反思与迭代范式引入了自我评估和改进机制。在生成初步结果后，LLM 对结果进行反思（如”这段代码是否正确实现了原函数的功能？”），发现潜在问题，然后进行迭代改进。这种范式特别适用于代码生成任务，能够逐步提高生成代码的质量。在本研究的驱动函数替换模块中，采用了类似的迭代机制：生成代码、编译验证、分析错误、修正代码，直到达到可编译状态。这些工作流范式为构建可靠的代码分析系统提供了设计模式。在实际应用中，可以根据任务的特点选择合适的范式，或者组合多种范式，以实现最佳效果。

LangChain 提供了统一的接口来调用不同的大语言模型，包括 OpenAI、Anthropic、智谱 AI、深度求索等多个提供商的模型。开发者可以轻松切换不同的模型，而无需修改应用代码。

（2）提示词模板（Prompt Templates）

提示词模板允许开发者定义可复用的提示词结构，通过变量注入实现动态生成。例如，可以定义一个代码分析模板，包含代码片段、分析要求等占位符，在运行时填入具体内容。

（3）链（Chains）

链是将多个组件按顺序组合的机制。例如，一个代码分析链可能包含：读取代码文件 → 构造提示词 → 调用 LLM → 解析结果 → 存储分析报告。通过链的组合，可以构建复杂的处理流程。

（4）代理（Agents）

代理是更高级的抽象，它允许 LLM 自主决定下一步行动。代理根据用户的输入，选择合适的工具（如搜索引擎、代码执行器、数据库查询），执行操作，并根据结果决定后续行动，直到完成任务。LangChain 内置了多种代理类型，如 ReAct 代理、OpenAI Functions 代理等。

（5）工具（Tools）

工具是代理可以调用的外部功能。LangChain 提供了大量内置工具，如网络搜索、Python 代码执行、文件操作等，开发者也可以自定义工具。工具的定义包括名称、描述、输入模式、执行函数等。

（6）记忆（Memory）

记忆组件允许应用在多次调用之间保持状态。例如，在多轮对话中，记忆组件可以存储历史对话，使 LLM 能够理解上下文。LangChain 提供了多种记忆实现，如缓冲记忆、摘要记忆、向量存储记忆等。

2.3.4.1 LangGraph 状态图

LangGraph 是 LangChain 生态中用于构建有状态、可循环的智能体工作流的库。与 LangChain 的链式结构不同，LangGraph 基于状态图（State Graph）的概念，支持更复杂的控制流。

（1）状态管理

LangGraph 的核心是状态（State）对象，它在工作流的各个节点之间传递和更新。状态可以包含任意类型的数据，如对话历史、中间结果、用户输入等。每个节点可以读取和修改状态，实现信息的流动和共享。

（2）节点和边

节点（Node）是工作流中的处理单元，接收状态作为输入，返回状态的更新。边（Edge）定义了节点之间的转换关系。LangGraph 支持条件边（Conditional Edge），根据状态的值动态选择下一个节点，实现分支逻辑。

（3）循环结构

与简单的链式结构不同，LangGraph 支持循环，允许工作流在满足特定条件前重复执行某些节点。这对于迭代优化、多轮交互等场景非常有用。例如，在代码生成任务中，可以设计一个循环：生成代码 → 编译验证 → 如果失败则修复 → 重新生成，直到代码可编译。

（4）持久化

LangGraph 支持检查点（Checkpoint）机制，可以将工作流的状态持久化存储。这使得工作流可以暂停和恢复，支持长时间运行的任务。在代码分析场景中，如果 LLM 调用超时或失败，可以从上一个检查点恢复，而不需要重新开始。

2.3.5 代码分析工作流范式

基于 LLM 智能体框架，可以构建多种代码分析工作流。以下是几种常见的工作流范式：

（1）ReAct 范式

ReAct（Reasoning + Acting）是一种结合推理和行动的工作流范式。在代码分析任务中，LLM 首先进行推理（”我需要理解这个函数的功能”），然后采取行动（”调用静态分析工具提取函数的调用关系”），观察结果，再进行下一步推理。这种范式使 LLM 能够动态地收集信息，做出决策，逐步完成复杂的分析任务。

（2）Plan-and-Execute 范式

Plan-and-Execute 范式将任务分解为规划和执行两个阶段。在规划阶段，LLM 分析任务要求，生成一个执行计划（如：1. 识别硬件相关代码；2. 分析驱动函数语义；3. 生成替代代码；4. 验证编译）。在执行阶段，按照计划依次执行各个步骤。这种范式的优点是结构清晰，易于调试和监控。

（3）Multi-Agent 范式

Multi-Agent 范式使用多个专门的智能体协作完成任务。例如，在驱动函数替换任务中，可以设计以下智能体：

- **分析智能体**：负责识别驱动函数，分析其语义和依赖关系；
- **生成智能体**：负责生成替代代码，考虑仿真环境的特点；
- **验证智能体**：负责编译验证，检查代码的正确性；
- **协调智能体**：负责协调各智能体的工作，处理冲突。

各智能体通过消息传递或共享状态进行协作，共同完成复杂的代码分析和生成任务。

（4）反思与迭代范式

反思与迭代范式引入了自我评估和改进机制。在生成初步结果后，LLM 对结果进行反思（”这段代码是否正确实现了原函数的功能？”），发现潜在问题，然后进行迭代改进。这种范式特别适用于代码生成任务，能够逐步提高生成代码的质量。在本研究的驱动函数替换模块中，采用了类似的迭代机制：生成代码 → 编译验证 → 分析错误 → 修正代码，直到达到可编译状态。

这些工作流范式为构建可靠的代码分析系统提供了设计模式。在实际应用中，可以根据任务的特点选择合适的范式，或者组合多种范式，以实现最佳效果。

2.4 程序分析技术

程序分析技术是软件工程和安全研究的重要基础，旨在通过自动化手段理解程序的行为和性质。在固件仿真研究中，程序分析技术用于识别代码中的硬件依赖、追踪数据流、建立函数调用关系，为后续的驱动函数替换提供准确的”硬件依赖画像”。

2.4.1 静态分析与动态分析

程序分析技术可分为静态分析和动态分析两大类。

静态分析（Static Analysis）在不执行程序的情况下，通过分析源代码或二进制代码来推断程序的属性。静态分析的优势在于能够覆盖程序的所有可能执行路径，不受测试用例的限制；其局限在于面临不可判定性问题（如停机问题），对于某些程序属性只能给出近似答案。常见的静态分析技术包括数据流分析、控制流分析、指针分析、抽象解释等。

动态分析（Dynamic Analysis）通过实际执行程序并观察其运行时行为来理解程序。动态分析能够获取精确的运行时信息（如变量值、内存状态、执行路径），但只能观察到被测试用例覆盖的部分，覆盖率不足可能导致遗漏重要行为。常见的动态分析技术包括动态插桩、污点追踪、符号执行等。

本研究主要采用静态分析技术。对于固件源码而言，静态分析具有以下优势：首先，源码层面的静态分析能够获取完整的程序信息（类型、符号、注释等），分析精度高于二进制层面的分析；其次，静态分析不需要真实硬件或仿真环境，可以在代码开发的早期阶段进行；第三，静态分析能够覆盖所有可能的执行路径，不会遗漏

任何驱动函数。

2.4.2 CodeQL 静态分析框架

CodeQL 是 GitHub 开发的语义级代码分析平台，支持多种编程语言（包括 C/C++、Java、Python、JavaScript 等）的安全漏洞检测和代码质量分析。CodeQL 的核心思想是将源代码转换为可查询的数据库，然后通过声明式查询语言（QL）进行灵活的分析。

2.4.2.1 CodeQL 数据库构建

CodeQL 数据库是源代码的结构化表示，包含语法树、控制流图、数据流图等多层次信息。数据库的构建过程分为源码准备、数据库创建和索引建立三个阶段。在源码准备阶段，需要将固件源代码组织为标准项目结构，确保包含所有头文件和依赖。对于使用构建系统（如 CMake、Makefile）的项目，CodeQL 可以利用构建信息进行更精确的分析。在数据库创建阶段，调用 CodeQL 命令行工具创建数据库，指定目标语言、源码路径和构建命令。在索引建立阶段，CodeQL 对 C/C++ 源文件建立多层次索引，包括抽象语法树（AST）表示源代码的语法结构，包含函数定义、变量声明、表达式等语法元素；控制流图（CFG）表示程序执行路径的有向图，表示基本块之间的跳转关系；数据流图（DFG）表示变量和表达式的数据传播关系，用于追踪数据的定义和使用；符号表与类型信息记录变量、函数、类型的定义和引用。

2.4.2.2 QL 查询语言

CodeQL 查询语言（QL）是一种声明式的逻辑编程语言，专门用于代码分析。QL 的语法类似于 Datalog，采用谓词逻辑的表达方式。查询的基本模式是定义实体（如函数、变量、表达式）之间的关系，然后通过逻辑连接构建复杂的分析规则。QL 查询的基本结构包括导入模块（如 `import cpp` 导入 C/C++ 分析库）、定义谓词（如 `isMMIOAccess(Expr e)` 判断表达式是否为 MMIO 访问）、定义类（用于封装相关的实体和谓词，支持继承和重写）、选择子句（定义查询的输出格式）。

2.4.2.3 数据流分析与污点追踪

数据流分析是 CodeQL 的核心能力之一，用于追踪数据在程序中的传播路径。污点追踪（Taint Tracking）是数据流分析的一种特殊形式，用于追踪”污点数据”（如用户输入、不可信数据）从源点（source）到汇点（sink）的传播路径。在 CodeQL 中，污点追踪通过配置类（Configuration）定义。源点（Source）定义污点数据的来源，如 MMIO 字段访问、外部输入函数等。汇点（Sink）定义污点数据的影响点，如函数参数、敏感操作等。污点步骤（Taint Step）定义污点数据在程序中的传播方式，如赋值、函数调用、指针解引用等。屏障（Barrier）定义阻断污点传播的语句，如数据清理、边界检查等。通过配置这些要素，CodeQL 能够自动追踪数据流路径，识别潜在的数据流问题。本研究利用 CodeQL 的污点追踪能力，建立 MMIO 访问与函数参数之间的关联关系。

2.5 固件仿真与模糊测试

固件仿真是物联网安全研究的重要技术手段。由于物联网设备的硬件多样性和封闭性，直接在真实设备上进行安全测试面临诸多困难。固件仿真技术通过软件模拟硬件环境，使研究人员能够在通用计算平台上运行和分析嵌入式固件，从而降低测试成本并提高效率。模糊测试作为一种高效的漏洞挖掘技术，与固件仿真相结合，能够在仿真环境中自动化地发现固件中的安全漏洞。

2.5.1 QEMU 仿真器

QEMU[9] 是目前应用最广泛的开源机器仿真器，支持多种处理器架构和外设仿真。QEMU 的设计目标是提供高性能、高可移植性的系统仿真能力，使其成为固件仿真研究的重要基础平台。

QEMU 支持两种主要的仿真模式。用户模式仿真（User Mode Emulation）允许在一种架构上运行为另一种架构编译的单个程序，通过动态二进制翻译技术将目标架构的指令转换为宿主机指令执行。用户模式仿真适用于分析用户空间程序，但由于物联网固件通常运行在裸机或嵌入式操作系统上，用户模式仿真在固件仿真领域的应用有限。

系统模式仿真（System Mode Emulation）提供完整的硬件系统仿真，包括处理器、内存、中断控制器、外设等组件。在系统模式下，QEMU 可以运行完整的操作系统或裸机固件，这是固件仿真的主要工作模式。QEMU 的系统模式仿真采用模块化设计，各种硬件组件以独立模块的形式实现，用户可以通过配置文件组合不同的组件来构建目标系统的硬件模型。

QEMU 的核心是动态二进制翻译引擎（Tiny Code Generator, TCG）。TCG 将目标架构的指令块翻译为中间表示（Intermediate Representation, IR），然后再将 IR 翻译为宿主机指令。这种两级翻译机制使 QEMU 能够支持多种目标架构和宿主架构的组合。为了提高性能，TCG 采用即时编译（JIT）技术，将翻译结果缓存在内存中供后续执行重复使用。

QEMU 提供了丰富的外设模型库，涵盖了常见的通信接口（UART、SPI、I2C）、存储设备（Flash、SD 卡）、网络设备（以太网控制器）等。对于物联网设备中常见的外设，QEMU 通常已经有相应的实现。然而，对于厂商自定义的专用外设，研究人员需要基于 QEMU 的外设框架自行实现仿真模型。这需要深入理解外设的硬件规范，工作量较大。

在固件仿真应用中，QEMU 还面临一些挑战。首先是仿真精度与性能的权衡：高精度的仿真需要模拟更多的硬件细节，但会带来显著的性能开销；简化的仿真模型性能更好，但可能与真实硬件行为存在偏差。其次是外设状态的一致性问题：外设的内部状态可能随时间变化，简单的状态机模型难以准确表达这种动态行为。此外，一些物联网设备使用非标准的启动流程和内存映射，需要对 QEMU 进行定制才能正确运行。

2.5.2 模糊测试技术

模糊测试（Fuzzing）是一种自动化的软件测试技术，通过向目标程序输入大量随机或经过变异的数据，触发程序的各种执行路径，从而发现潜在的漏洞和缺陷。模糊测试技术具有自动化程度高、测试覆盖面广、能够发现未知漏洞等优点，在软件安全领域得到了广泛应用。

模糊测试的基本流程包括以下几个步骤：首先，种子选择模块从初始种子集合

中选择一个种子作为变异基础；然后，变异模块对种子进行随机变异或基于规则的变异，生成新的测试用例；接着，测试执行模块将测试用例输入目标程序并监控其执行；最后，结果分析模块根据执行结果判断是否触发了异常行为（如崩溃、断言失败等），并将有价值的测试用例加入种子集合供后续变异使用。

模糊测试技术的发展可以分为几个阶段。第一代模糊测试采用黑盒方法，通过生成随机输入数据来测试程序。这种方法实现简单，但由于缺乏对程序结构的了解，测试效率较低，难以触发深层代码路径。第二代模糊测试引入了反馈机制，通过代码插桩等技术获取程序执行路径信息，利用这些信息指导测试用例的生成，代表工具包括 AFL、libFuzzer 等。这种灰盒方法显著提高了测试效率和覆盖率。第三代模糊测试结合了符号执行、污点分析等技术，能够更精确地分析程序约束，生成满足特定路径条件的测试用例，代表工具包括 Driller、QSYM 等。

代码覆盖率是衡量模糊测试效果的重要指标。覆盖率引导的模糊测试（Coverage-guided Fuzzing）在测试过程中持续监控代码覆盖率，优先保留能够触发新执行路径的测试用例，从而逐步扩大测试范围。这种策略使得模糊测试能够在有限的测试时间内探索更多的程序状态，提高漏洞发现的概率。

2.5.3 嵌入式固件模糊测试

嵌入式固件模糊测试是物联网安全研究的重要方向，其核心目标是在仿真环境中对固件进行自动化安全测试。根据测试执行环境的不同，固件模糊测试方法可以分为两类。

（1）硬件在环方法

硬件在环（Hardware-in-the-Loop, HIL）方法在真实硬件上运行固件，通过专用接口（如 JTAG、SWD、串口等）注入测试数据。这种方法能够保证与真实硬件的一致性，测试结果可靠。然而，硬件在环方法也存在明显的局限性：测试速度受限于硬件速度，难以进行大规模并发测试；需要物理访问目标设备，对于已部署的设备难以实施；每次测试后可能需要手动复位设备，自动化程度有限。

（2）基于仿真的方法

基于仿真的方法在软件仿真器（如 QEMU）中运行固件，通过软件接口注入测

试数据。这种方法具有测试速度快、易于自动化、可并发执行等优点，是当前研究的主流方向。基于仿真的方法需要解决的核心问题是固件在仿真环境中的硬件依赖：仿真器需要为固件提供足够的外设支持，使固件能够正常运行到测试目标点。针对这一问题，研究人员提出了多种外设建模方法，包括手动建模、基于硬件追踪的自动建模、基于程序分析的推断建模等。

2.5.4 AFL 及其变体

American Fuzzy Lop（AFL）[39] 是最具影响力的覆盖率引导模糊测试工具之一，其设计思想和技术实现对后续的模糊测试工具产生了深远影响。

AFL 的核心是轻量级的代码插桩和高效的变异策略。在编译时插桩模式下，AFL 通过修改编译器（如 afl-gcc、afl-clang）在程序中插入覆盖率记录代码；在即时插桩模式下，AFL 利用 QEMU 的动态翻译功能在运行时插入插桩代码。无论哪种模式，插桩代码都会记录程序执行过程中经过的基本块和边，这些信息用于计算覆盖率并指导测试用例的选择。

AFL 维护一个测试用例队列，初始包含用户提供的种子文件。在测试过程中，AFL 不断从队列中选择测试用例，对其进行变异，执行变异后的测试用例，并根据覆盖率变化决定是否将新用例加入队列。AFL 的变异策略包括位翻转、字节删除、字节插入、块拼接、算术运算等多种操作，能够产生多样化的测试输入。通过这种迭代式的测试和筛选，AFL 能够逐步探索程序的各种执行路径。

AFL 的成功催生了大量改进和变体工具。AFL++ 是 AFL 的增强版本，提供了更丰富的变异策略、更好的并行支持和可扩展的插件架构。libFuzzer 是 LLVM 项目提供的库式模糊测试框架，与 AFL 的进程级模糊测试不同，libFuzzer 将模糊测试逻辑嵌入目标程序内部，以函数调用的方式执行测试，效率更高。这些工具各有特点，适用于不同的测试场景。

在嵌入式固件模糊测试领域，研究人员基于 AFL 开发了多种专用工具。这些工具通常结合固件仿真技术，在仿真环境中运行固件并注入测试数据。例如，Firmadyne、FirmAE 等工具针对路由器固件进行模糊测试；Fuzzware、P2IM 等工具针对嵌入式固件的外设建模问题，通过自动生成外设模型来支持固件在仿真环境中

的运行。这些工作展示了模糊测试技术在物联网安全领域的应用潜力。

2.6 本章小结

本章介绍了物联网设备固件仿真研究涉及的基础知识，包括物联网设备硬件基础（ARM Cortex-M 架构、MMIO、中断机制）、物联网操作系统与驱动（RTOS、HAL、驱动函数分类）、大模型代码分析与生成技术（Transformer、GPT、CodeBERT、智能体工作流）以及固件仿真与模糊测试技术（QEMU、AFL），为后续章节的论述奠定基础。

3 系统设计与实现

本章详细阐述基于驱动程序替换的固件仿真系统的设计与实现。系统采用”静态识别 → 智能替换 → 构建验证 → 动态反馈”的分层架构，通过 CodeQL 静态分析和大语言模型智能体技术，实现固件驱动函数的自动识别、语义分析和代码替换，最终在仿真环境中运行并支持模糊测试。

3.1 整体架构设计

3.1.1 设计目标与核心思想

物联网设备固件的安全测试面临硬件依赖性强、外设建模成本高、测试环境搭建困难等挑战。本研究提出的”基于驱动程序替换的固件仿真技术”旨在解决上述问题，其核心目标是：在缺少真实硬件外设、外设建模成本过高或外设规格不可得的情况下，使固件仍可在通用仿真环境中稳定运行，并进一步支持自动化测试与模糊测试。

核心思想是将固件中与具体硬件强绑定的驱动代码（包括寄存器读写、HAL/外设库调用、中断与 DMA 配置等）识别出来，并用可在仿真环境中执行的替代实现进行等价替换，从而实现”固件逻辑与硬件依赖解耦”。这种方法的本质是在源码层面进行程序变换，将硬件依赖代码替换为与仿真环境兼容的等价实现，而非在硬件层面进行外设建模。

具体而言，系统设计需要满足以下目标：

传统固件仿真方法需要安全研究人员手工分析固件代码，理解硬件交互逻辑，编写外设模型或驱动替代代码，这一过程耗时且容易出错。本系统通过静态分析和大语言模型技术，尽量减少人工干预，实现从源码到可仿真固件的端到端自动化流程，确保自动化程度高。

物联网设备的硬件平台和软件架构具有高度多样性。硬件平台涵盖 STM32、NXP、Nordic 等厂商的多种 MCU 芯片，软件架构包括 FreeRTOS、RT-Thread、Zephyr 等实时操作系统以及无操作系统的裸机程序。系统设计需要支持多种硬件平台和操

作系统，具有良好的可扩展性。

驱动函数替换的核心挑战在于如何保证替换后的代码在语义上与原代码等价。替换后的代码应保留原有业务逻辑，正确维护驱动状态，避免引入新的缺陷。系统需要通过静态验证和动态测试双重保障替换的语义正确性，确保语义保持性好。

代码替换是一个迭代优化的过程，初始生成的替换代码可能存在编译错误或运行时问题。系统需要建立完整的反馈闭环，通过动态测试验证替换效果，并将反馈信息用于指导代码修复和重新生成，确保反馈闭环完整。

3.1.2 系统分层架构

系统采用分层架构设计，按”静态识别 → 智能替换 → 构建验证 → 动态测试反馈”的链路组织为四个核心模块，如图 3.1所示。每一层模块承担明确的职责，通过标准化的数据接口进行交互，形成完整的处理流水线。

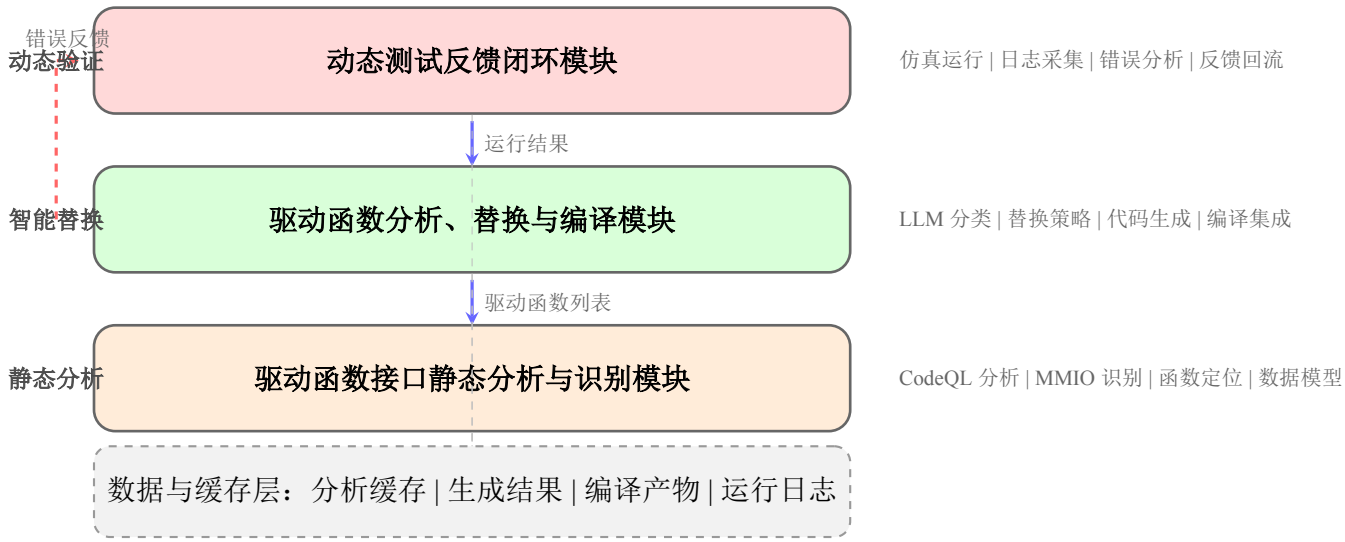

图 3.1 系统整体架构图

架构的顶层是**动态测试反馈闭环模块**，负责在仿真环境中运行替换后的固件，采集运行日志、崩溃信息与覆盖率等指标，并将失败样例与错误信息结构化回流，触发重新分析与再生成。该模块是系统质量保障的关键环节。

中间层是**驱动函数分析、替换与编译模块**，这是系统的核心智能层。该模块基于上游静态分析结果，使用大语言模型对驱动函数进行类型分类与语义保留分析，

生成可替代实现，同时将替代实现集成回工程并驱动构建。该模块采用 LangGraph 智能体框架实现复杂的推理和决策流程。

底层是**驱动函数接口静态分析与识别模块**，这是整个方案的基础支撑层。该模块对固件源码进行静态分析，定位硬件相关代码，抽取驱动函数、MMIO 访问点、依赖数据结构、调用关系等信息，并以统一的数据模型对外提供查询接口。模块基于 CodeQL 静态分析引擎实现，通过预定义的查询规则从固件源码中提取驱动相关信息。

贯穿上述三层的是**数据与缓存层**，统一管理静态分析缓存、生成结果缓存、编译产物与运行日志，支持断点续跑和结果复用。该层确保系统在长时间运行过程中能够保存中间状态，避免重复计算，提高处理效率。

3.1.3 核心模块职责

系统架构中的四个核心模块各司其职，协同工作，共同完成从固件源码到可仿真固件的自动化转换。本节详细阐述各模块的职责定位、设计考量和技术挑战。

3.1.3.1 驱动函数接口静态分析与识别模块

驱动函数接口静态分析与识别模块是整个系统的”感知器官”，负责从固件源码中提取硬件依赖信息，构建固件的”硬件依赖画像”。该模块的设计面临着嵌入式代码的多样性和复杂性挑战：不同厂商的 HAL 库采用不同的抽象层次和编程风格，寄存器访问模式千差万别，驱动函数与业务逻辑的边界往往模糊不清。

模块的核心职责是建立从源码到硬件依赖信息的映射。具体而言，模块需要回答以下关键问题：固件中哪些函数与硬件交互？这些函数通过何种方式访问硬件（直接寄存器访问、HAL 库调用、还是间接的句柄操作）？函数之间是否存在调用依赖关系？这些信息构成了后续智能替换的基础，其准确性直接决定了替换的质量和可靠性。

为实现这一目标，模块采用 CodeQL 静态分析引擎作为底层支撑。CodeQL 通过构建包含语法树、控制流图、数据流图的综合数据库，为语义级代码分析提供了强大的查询能力。模块首先为固件源码构建 CodeQL 数据库，然后通过精心设计的查

询规则识别 MMIO 结构体实例和字段访问，进而对涉及硬件访问的函数进行语义分类。数据流追踪是该模块的关键技术，它通过污点追踪建立寄存器访问与函数参数之间的关联，为替换模块提供精确的语句定位信息。

模块输出的”硬件依赖画像”不仅包含驱动函数列表和 MMIO 访问点，还包括函数调用关系图和数据流信息。这些结构化的分析结果被存储在统一的数据模型中，为下游模块提供可查询的知识库。

3.1.3.2 驱动函数分析、替换与编译模块

驱动函数分析、替换与编译模块是系统的”大脑”，承担着语义理解和代码生成的核心任务。如果说静态分析模块解决了”是什么”的问题（哪些代码与硬件相关），那么替换模块需要解决”怎么做”的问题（如何生成语义等价的替代代码）。

模块面临的核心挑战是语义保持性。驱动函数的替换不能简单地进行代码删除或占位，而需要在保留业务逻辑的同时，将硬件依赖部分替换为与仿真环境兼容的实现。例如，一个 UART 发送函数不仅包含寄存器写入操作，还包含状态管理、错误处理、超时控制等业务逻辑，替换时需要准确识别并保留这些逻辑。这要求替换模块具备深层的代码理解能力，能够区分”必须替换的硬件操作”和”必须保留的业务逻辑”。

为实现这一目标，模块采用大语言模型驱动的智能体架构。与传统基于规则或模板的代码生成方法不同，LLM 具备理解代码语义的能力，能够根据函数的具体实现和上下文生成定制化的替换代码。模块使用 LangGraph 框架构建图式工作流，通过”推理-工具调用-结果汇总”的迭代过程，逐步完成函数分类、策略选择、代码生成和验证等工作。

模块设计了四种替换策略（Skip、Strip、Redirect、Event），每种策略适用于特定类型的驱动函数。策略的选择不是硬编码的规则匹配，而是由 LLM 根据函数的语义特征自主决定。生成替换代码后，模块通过语法检查和语义规则检查（Rubric）进行质量验证，确保替换代码的可靠性。

编译集成是模块的最后一个环节。模块将替换代码写回源文件，调用构建系统

执行编译，并处理可能出现的编译错误。当编译失败时，模块通过 Fixer Agent 进行自动修复，形成”编译-错误解析-修复-重新编译”的闭环，直至编译成功。

3.1.3.3 程序分析与动态测试反馈闭环模块

程序分析与动态测试反馈闭环模块是系统的”验证官”，负责在仿真环境中检验替换效果，并将发现的问题反馈给上游模块。该模块体现了软件工程中”测试驱动”的思想：静态分析和代码替换的正确性最终需要通过实际运行来验证。

模块的职责包括仿真环境搭建、固件运行监控和反馈信息结构化三个方面。仿真环境搭建涉及 Unicorn 引擎的配置，包括选择合适的 CPU 架构模式、配置内存映射和寄存器参数等。为支持替换后的驱动代码与仿真环境交互，模块设计了 LCMHal（Lightweight Communication and Hardware Abstraction Layer）虚拟外设接口层。LCMHal 定义了统一的外设操作接口（init、read、write、ioctl、deinit），每种外设类型实现该接口并注册到框架中。替换后的驱动代码通过调用 LCMHal 接口与仿真环境交互，而不直接访问硬件寄存器。

固件运行监控是该模块的核心任务。模块通过 Emulator Runner Agent 执行仿真，实时采集运行日志、覆盖率数据和异常信息。监控的重点包括程序崩溃（如段错误、断言失败）、运行超时（如死循环、阻塞等待）、以及功能异常（如输出结果与预期不符）。当检测到问题时，模块需要对原始日志进行结构化处理，提取关键信息（错误类型、异常地址、关联函数、调用栈等），生成反馈摘要。

反馈闭环是该模块的关键设计。当运行失败时，反馈摘要被传递给 Fixer Agent，触发重新分析或修复流程。Fixer Agent 根据错误类型和位置，定位问题函数，生成修复代码，然后重新进入”替换-编译-运行”的循环。这个闭环机制确保系统能够通过迭代优化不断提高替换质量，直到固件能够在仿真环境中稳定运行。

3.1.3.4 数据与缓存层

数据与缓存层是系统的”记忆系统”，贯穿整个处理流程，提供数据持久化和缓存服务。虽然该层不直接参与核心的代码分析和替换工作，但其对于系统的可靠性、

效率和可维护性至关重要。

固件仿真是一个计算密集型的过程，涉及静态分析、LLM 推理、编译构建等多个耗时环节。如果每次运行都从头开始，不仅效率低下，而且成本高昂。数据与缓存层通过缓存中间结果，支持断点续跑和增量处理。例如，CodeQL 数据库的构建通常需要数分钟，缓存后后续查询可以直接复用；LLM 生成的替换代码和分类结果被缓存后，当处理相同函数时可以直接使用，无需重复调用模型。

该层管理的数据包括四个类别：静态分析缓存（CodeQL 数据库、查询结果、函数信息等）、生成结果缓存（LLM 分类结果、替换代码、验证报告等）、编译产物管理（不同版本的 ELF 文件、构建日志等）和运行日志存储（仿真运行日志、崩溃信息、覆盖率数据等）。所有数据采用统一的命名规范和目录结构，支持按项目、按任务、按时间戳进行组织。

数据与缓存层还承担着系统可观测性的职责。通过分析缓存数据，可以追溯系统的处理历史，了解每个固件经历了哪些分析、替换和修复步骤，这对于问题诊断和系统优化具有重要价值。

3.1.4 数据流与控制流

系统的数据流从源码出发，经静态分析产生”硬件依赖画像”，随后驱动替换模块在该画像约束下生成替代代码并集成构建，最后由动态执行产生反馈数据回流。整个数据流是有向的、可追溯的，每个处理阶段的输出都成为下一阶段的输入。

关键控制流以”任务”（Task）为单位组织。每个任务对应一个固件工程、一个构建配置与一组待替换函数列表。系统对每个任务维持明确的状态机，状态转移如图 3.4所示。

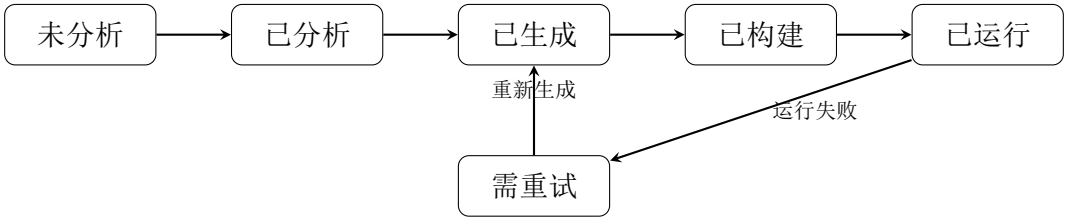

图 3.2 任务状态机

状态机的初始状态为”未分析”，表示任务已创建但尚未开始处理。经过静态分析后进入”已分析”状态，此时驱动函数已被识别和分类。智能体完成代码生成后进入”已生成”状态，替换代码已生成并验证。编译构建成功后进入”已构建”状态，ELF 文件已生成。仿真运行完成后进入”已运行”状态，表示任务成功完成。

如果在运行阶段检测到问题（如运行崩溃、功能异常），任务进入”需重试”状态，系统会根据反馈信息重新分析问题函数，生成新的替换代码，然后重新进入”已生成”状态，开始新一轮的构建和测试。这个闭环机制确保系统能够通过迭代优化不断提高替换质量。

3.2 驱动函数接口静态分析与识别模块

本模块是整个方案的基础支撑层，负责自动、精准地定位固件中与硬件交互的代码，并将其抽象为标准化的”可查询知识库”。模块基于第二章介绍的 CodeQL 静态分析框架实现，通过预定义的查询规则从固件源码中提取驱动相关信息。本节重点阐述 MMIO 访问模式识别、驱动函数分类和数据流追踪的具体实现方法。

3.2.1 MMIO 访问模式识别

MMIO 访问模式识别是驱动函数分析的首要任务，其目标是定位代码中所有与硬件寄存器交互的位置。识别的准确性直接影响后续驱动函数分类和替换的质量。本研究采用”结构体定位 → 字段访问 → 函数关联”的三阶段识别方法。

3.2.1.1 MMIO 实例定位方法

MMIO（Memory-Mapped I/O）结构体是嵌入式厂商定义的寄存器映射结构，每个字段对应一个硬件寄存器，字段的偏移量与硬件规范中的寄存器地址偏移一致。定位 MMIO 结构体实例是识别硬件相关代码的关键步骤，本研究总结出三种典型的定位模式。在此之前，首先阐述 MMIO 结构体的识别特征。

（1）MMIO 结构体识别特征

MMIO 结构体具有三个典型特征。命名模式上，MMIO 结构体通常采用规范化的命名模式，如 STM32 HAL 库使用 `_TypeDef` 后缀（`USART_TypeDef`、`GPIO_TypeDef`），

NXP SDK 使用 _Type 后缀（USART_Type、GPIO_Type）。字段类型上，字段类型为 volatile uint32_t 或类似的 volatile 整型，确保编译器不优化对寄存器的访问。常见的宏定义包括 __IO（读写）、__I（只读）、__O（只写）。字段命名上，字段名通常为寄存器缩写，遵循厂商的命名规范，例如控制寄存器命名为 CR1、CR2、CR3，状态寄存器命名为 SR、ISR，数据寄存器命名为 DR、TDR、RDR，配置寄存器命名为 BRR（波特率）、DIER（中断使能）等。

典型的 MMIO 结构体定义如下所示，以 STM32 HAL 库中的 USART 外设为例：

```
// STM32 HAL 库中的典型 MMIO 结构体
typedef struct
{
  __IO uint32_t CR1;    /*!< Control register 1,   */
  __IO uint32_t CR2;    /*!< Control register 2,   */
  __IO uint32_t CR3;    /*!< Control register 3,   */
  __IO uint32_t BRR;    /*!< Baud rate register,   */
  __IO uint32_t GTPR;   /*!< Guard time register,  */
  __IO uint32_t RTOR;   /*!< Receiver Time Out,    */
  __IO uint32_t RQR;    /*!< Request register,     */
  __IO uint32_t ISR;    /*!< Interrupt status,     */
  __IO uint32_t ICR;    /*!< Interrupt clear,      */
  __IO uint32_t RDR;    /*!< Receive Data,         */
  __IO uint32_t TDR;    /*!< Transmit Data,        */
} USART_TypeDef;
```

该结构体定义了 USART 外设的所有寄存器，每个字段的偏移量与硬件手册中的寄存器地址偏移完全一致。例如，CR1 字段位于结构体起始位置（偏移 0），对应硬件地址 USART_BASE + 0x00；DR 字段位于偏移 0x1C 处，对应硬件地址 USART_BASE + 0x1C。这种一一映射关系是 MMIO 访问的基础。

（2）MMIO 实例定位模式

（1）MMIO 结构体特征

MMIO（Memory-Mapped I/O）结构体是嵌入式厂商定义的寄存器映射结构，每个字段对应一个硬件寄存器，字段的偏移量与硬件规范中的寄存器地址偏移一致。正确识别 MMIO 结构体及其实例是驱动函数分析的基础。本研究通过以下特征识别 MMIO 结构体：

命名模式：MMIO 结构体通常采用规范化的命名模式，不同厂商和 SDK 有不同的命名约定。STM32 HAL 库使用 _TypeDef 后缀（如 USART_TypeDef、GPIO_TypeDef、SPI_TypeDef），NXP SDK 使用 _Type 后缀（如 USART_Type、GPIO_Type），Nordic

nRF5 SDK 也使用 _Type 后缀（如 NRF_UART_Type、NRF_GPIO_Type）。这种命名模式并非强制要求，但大多数厂商都遵循类似的约定，便于代码阅读和维护。

字段类型：所有字段均为 volatile 整型（通常是 uint32_t），确保编译器不优化对寄存器的访问。volatile 关键字告诉编译器该变量可能被程序外部修改（如硬件状态变化），因此每次访问都必须从内存读取，不能使用缓存值。常见的 volatile 宏定义包括 __IO（读写寄存器）、__I 或 __IM（只读寄存器，输入模式）、__O 或 __OM（只写寄存器，输出模式）。这些宏定义在厂商提供的头文件中，展开后通常包含 volatile 关键字。

字段命名：字段名通常为寄存器缩写，遵循厂商的命名规范。常见的寄存器命名模式包括：控制寄存器（CR1、CR2、CR3，用于配置外设工作模式）、状态寄存器（SR、ISR，用于查询外设状态和中断标志）、数据寄存器（DR、TDR、RDR，用于数据发送和接收）、配置寄存器（BRR 波特率配置、DIER DMA/中断使能、CFGR 时钟配置）、DMA 寄存器（DMABASE DMA 基址、DMACNDAR DMA 计数器）。这些命名模式并非绝对，不同厂商可能有不同的命名约定，但总体遵循”功能缩写 + 数字编号”的模式。

（2）MMIO 实例定位模式

基于上述结构体特征，本研究总结出三种典型的 MMIO 实例定位模式：

模式一：直接地址强转

将固定地址强制转换为外设结构体指针，这是最直接的方式。该模式常见于寄存器定义的头文件中：

```
#define USART1_BASE     0x40011000U
#define USART1          ((USART_TypeDef *)USART1_BASE)

// 使用时直接通过宏访问
USART1->DR = data;  // 写入数据寄存器
status = USART1->SR;  // 读取状态寄存器
```

识别特征：存在整数常量到结构体指针的强制类型转换，地址常量位于外设地址范围（如 STM32 的 0x40000000-0x5FFFFFFF）。

模式二：间接初始化

通过配置表或描述符将外设基地址赋值给句柄，常见于 HAL 库的句柄结构：

```
typedef struct {
    USART_TypeDef *Instance;  // 外设基址
```

```
    UART_InitTypeDef Init;     // 配置参数
    HAL_LockTypeDef Lock;      // 锁状态
    // ... 其他字段
} UART_HandleTypeDef;

UART_HandleTypeDef huart1;
huart1.Instance = USART1;  // 间接赋值
```

识别特征：存在结构体指针类型的成员变量（通常命名为 Instance），该成员被赋值为外设基地址宏。

模式三：常量指针参数

函数以常量指针形式接收外设基址，常见于底层驱动函数：

```
void HAL_UART_Transmit(USART_TypeDef *huart,
                       uint8_t *pData, uint16_t Size);
```

识别特征：函数参数类型为 MMIO 结构体指针，函数内部通过该参数访问寄存器字段。

三种定位模式的对比如表 3.1所示。

表 3.1 MMIO 结构体实例定位模式对比

模式	描述	识别特征
直接地址强转	将固定地址强制转换为外设结构体指针	整数常量 → 结构体指针类型转换
间接初始化	通过配置表将外设基地址赋值给句柄	成员变量赋值为外设宏
常量指针参数	函数以常量指针形式接收外设基址	参数类型为 MMIO 结构体指针

3.2.1.2 CodeQL 查询实现

针对上述三种定位模式，本研究设计了相应的 CodeQL 查询规则。以” 直接地址强转” 模式为例，查询的核心逻辑如下：

```
/**
 * MMIO 结构体实例定位查询
 * 识别代码中通过地址强转方式获取的外设指针
 */
import cpp

from Variable v, Type t, Expr init
where
  // 类型名以 _TypeDef 或 _Type 结尾
```

```
  t.getName().regexpMatch(".*_TypeDef|.*_Type") and
  // 变量类型为指向该结构体的指针
  v.getType().(PointerType).getBaseType() = t and
  // 获取初始化表达式
  init = v.getInitializer() and
  // 初始化涉及地址常量或宏
  exists(MacroInvocation mi |
    init = mi.getExpr() and
    mi.getMacroName().regexpMatch(".*_BASE")
  )
select v, t, init, "MMIO instance: " + v.getName()
```

该查询从 CodeQL 数据库中提取所有符合 MMIO 结构体特征的变量及其初始化表达式。对于”间接初始化”和”常量指针参数”模式，采用类似的查询策略，通过追踪赋值语句和函数参数来识别 MMIO 实例。

3.2.2 驱动函数定位准则

静态分析模块的核心任务是识别固件中的驱动函数，为后续的智能替换提供目标列表。本研究采用简单直接的定位准则：**凡是包含 MMIO 访问的函数，均定义为驱动函数**。

这一定位准则基于以下考虑：

（1）硬件交互的直接性：MMIO 访问是固件与硬件交互的最底层方式，包含寄存器读写的函数必然与硬件强相关，需要在仿真环境中进行替换处理。

（2）分析的可靠性：MMIO 访问可以通过静态分析方法准确识别（如前文所述的结构体特征和定位模式），分析结果可靠且可验证。

（3）替换的必要性：对于不包含 MMIO 访问的函数，通常是纯软件逻辑（数据处理、协议解析、状态管理等），在仿真环境中可以直接运行，无需替换。

基于这一定位准则，驱动函数识别的流程如下：

步骤一：识别 MMIO 结构体实例

通过前文所述的三种定位模式（直接地址强转、间接初始化、常量指针参数），识别代码中的所有 MMIO 结构体实例。记录每个实例的变量名、类型、作用域和初始化位置。

步骤二：追踪 MMIO 字段访问

对每个 MMIO 结构体实例，追踪其字段访问（field access）表达式。记录访问的字段名、访问类型（读/写）、所在语句和所在函数。

步骤三：汇总驱动函数列表

对所有包含 MMIO 字段访问的函数进行汇总，形成驱动函数列表。列表中的每个条目包含函数名、位置、签名、MMIO 访问点列表等信息。

步骤四：构建函数调用关系

分析驱动函数之间的调用关系，构建函数调用图。调用关系用于后续的替换策略选择：如果函数 A 调用函数 B，且两者都是驱动函数，需要考虑替换的一致性。

驱动函数定位的形式化定义如下：

设 F 为固件中的函数集合，M 为 MMIO 结构体实例集合，$A(f, m)$ 为谓词" 函数 f 访问 MMIO 实例 m 的字段"，则驱动函数集合 D 定义为：

$$D = \{f \in F \mid \exists m \in M : A(f, m)\}$$

该定义确保所有与硬件交互的函数都被纳入驱动函数集合，为后续的语义分类和代码替换提供完整的候选集。需要注意的是，驱动函数的语义分类（如初始化、发送、接收等）不是由静态分析模块完成的，而是由后续的 LLM 智能体根据函数源码和上下文进行判断。静态分析模块只负责" 定位"，不负责" 分类"。

3.2.3 数据流追踪

3.2.3.1 追踪目标

数据流追踪是静态分析的重要环节，其目标是建立 MMIO 访问与函数接口（参数、返回值）之间的关联关系。本模块以"MMIO 字段访问" 为源点（source），以" 函数参数、局部变量、返回值、关键分支条件" 为汇点（sink），构建数据流配置并追踪路径。

数据流追踪的目的包括：

（1）确定参数影响：识别哪些函数参数最终影响了 MMIO 访问。例如，在 `HAL_UART_Transmit` 函数中，`pData` 参数通过数据流影响 `DR` 寄存器的写入值。

（2）识别逻辑边界：区分需要保留的业务逻辑与需要替换的硬件操作。例如，在发送函数中，状态变量更新是业务逻辑需要保留，而寄存器写入是硬件操作需要替

换。

（3）提供定位信息：为替换模块提供精确的语句定位信息，包括需要替换的代码行号、涉及的变量和表达式等。

3.2.3.2 数据流分析实现

CodeQL 提供了强大的数据流分析框架，支持污点追踪（Taint Tracking）和数据流追踪（Data Flow）。本研究利用该框架实现 MMIO 数据流追踪。

数据流配置的定义如下：

```
/**
 * MMIO 数据流追踪配置
 * 从寄存器访问追踪到函数参数
 */
import cpp, semmle.code.cpp.dataflow.TaintTracking

class MMIOAccessConfig extends TaintTracking::Configuration {
  MMIOAccessConfig() { this = "MMIOAccessConfig" }

  // 定义源点: MMIO 结构体字段访问
  override predicate isSource(DataFlow::Node source) {
    exists(FieldAccess fa |
      // 字段名匹配典型的寄存器名称
      fa.getTarget().getName().regexpMatch("DR|SR|CR.*|ISR|ICR") and
      source.asExpr() = fa
    )
  }

  // 定义汇点: 函数参数
  override predicate isSink(DataFlow::Node sink) {
    sink.asParameter().getFunction().getName().regexpMatch("HAL_.*")
  }

  // 定义额外的污点步骤: 通过指针解引用传播
  override predicate isAdditionalTaintStep(
    DataFlow::Node pred, DataFlow::Node succ
  ) {
    exists(PointerDereferenceExpr pde |
      succ.asExpr() = pde and
      pred.asExpr() = pde.getOperand()
    )
  }
}
```

该配置定义了从 MMIO 字段访问（源点）到 HAL 函数参数（汇点）的污点追踪路径。通过执行该查询，可以获得数据流路径，用于指导后续的代码替换。

3.2.4 统一数据模型

为便于后续模块使用，静态分析结果被组织为统一的数据模型。数据模型采用 Python 数据类（dataclass）定义，支持序列化和反序列化。

（1）函数信息模型

```
@dataclass
class FunctionInfo:
    """函数基本信息"""
    name: str                       # 函数名
    file_path: str                  # 所在文件路径
    line_start: int                 # 起始行号
    line_end: int                   # 结束行号
    signature: str                  # 函数签名
    return_type: str                # 返回类型
    parameters: List[Parameter]     # 参数列表
    category: str                   # 函数分类（RECV/SEND/IRQ/...）
    source_code: str                # 函数源码
    caller_functions: List[str]     # 调用该函数的函数列表
    callee_functions: List[str]     # 该函数调用的函数列表
```

（2）MMIO 访问点模型

```
@dataclass
class MMIOAccess:
    """MMIO 访问点"""
    struct_type: str                # 结构体类型（如 USART_TypeDef）
    field_name: str                 # 字段名（如 DR, SR）
    access_type: str                # 访问类型（read/write）
    line_number: int                # 行号
    enclosing_function: str         # 所在函数
    context_code: str               # 上下文代码片段
```

（3）项目分析结果模型

```
@dataclass
class ProjectAnalysisResult:
    """项目分析结果"""
    project_path: str                   # 项目路径
    codeql_db_path: str                 # CodeQL 数据库路径
    functions: List[FunctionInfo]       # 所有识别的函数
    mmio_accesses: List[MMIOAccess]     # 所有 MMIO 访问点
    call_graph: Dict[str, List[str]]    # 函数调用图
    driver_functions: List[str]         # 驱动函数列表（需要替换）
    analysis_timestamp: str             # 分析时间戳
```

这些数据模型通过 JSON 序列化存储在缓存层，供后续模块查询和使用。

3.3 驱动函数分析、替换与编译模块

本模块是系统的核心智能层，负责基于静态分析结果对驱动函数进行语义理解和代码替换。模块采用大语言模型（LLM）驱动的智能体（Agent）架构，通过工具调用和工作流编排实现自动化的函数分析与替换。

3.3.1 智能体工作流设计

3.3.1.1 LangGraph 工作流框架

系统采用 LangGraph 框架构建图式工作流。LangGraph 是 LangChain 生态中用于构建有状态、可循环的智能体工作流的库，支持复杂的状态管理和条件分支。

智能体工作流的核心包含三个逻辑节点，如图 3.3所示：

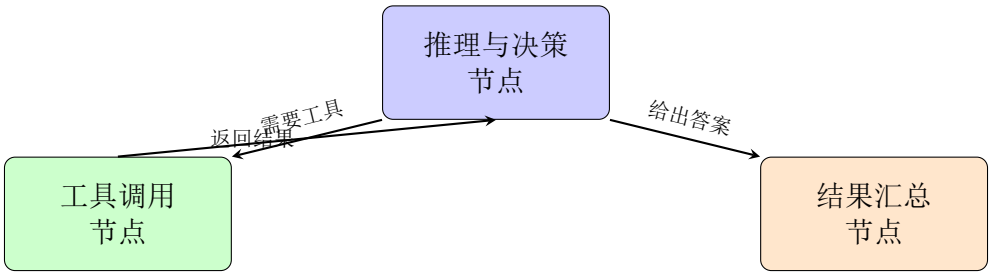

图 3.3 智能体工作流架构图

推理与决策节点：这是智能体的核心，由大语言模型驱动。该节点负责分析驱动函数的源码，判断函数类别，选择替换策略，决定是否需要调用工具获取补充信息。当 LLM 认为已有足够信息时，输出最终的分类和替换结果。

工具调用节点：当推理节点需要补充信息时，会请求调用工具。系统提供多种工具，包括：查询函数源码、获取调用关系、查询 MMIO 访问点、获取函数签名等。工具执行完毕后，结果返回给推理节点，继续推理。

结果汇总节点：当推理节点给出最终答案时，进入结果汇总节点。该节点对 LLM 的输出进行结构化解析，验证输出格式，将结果写入缓存。

3.3.1.2 状态机设计

LangGraph 的核心是状态（State）对象，它在工作流的各个节点之间传递和更新。状态机的设计需要考虑智能体在执行过程中需要维护的信息，以及状态之间的转移逻辑。本研究设计的状态模型和状态转移如下所述。

状态模型需要维护智能体执行过程中的关键信息。对话历史（messages）记录 LLM 与工具之间的所有交互，支持多轮对话的上下文保持。最终响应（final_response）存储结构化的分类结果或替换代码，类型为可选的 FunctionClassifyResponse 或 ReplacementResponse。当前处理的函数名（function_name）标识正在分析的函数，用于日志记录和结果关联。重试次数（retry_count）记录当前任务的迭代次数，用于防止无限循环。状态模型的 Python 定义如下：

```
class AgentState(MessagesState):
```

```
    """智能体状态模型"""
    messages: List[BaseMessage]            # 对话历史
    final_response: Optional[Response]     # 最终响应
    function_name: str                     # 当前函数名
    retry_count: int = 0                   # 重试次数
    max_retries: int = 3                   # 最大重试次数
```

状态转移逻辑控制智能体的执行流程。智能体从 START 状态开始，首先进入推理节点（agent），LLM 根据当前状态决定下一步行动。如果 LLM 请求调用工具（如查询函数源码、获取调用关系），则转移到工具节点（tools），工具执行完毕后返回推理节点继续处理。如果 LLM 认为已经获得足够信息并给出最终答案，则转移到结果节点（respond），对输出进行结构化解析后进入 END 状态。如果在执行过程中发生错误（如工具调用失败、输出格式错误），且重试次数未超过上限，则返回推理节点重新尝试；否则进入错误处理流程。状态转移的伪码描述如下：

```
Algorithm: Agent State Transition
Input: initial_state with function_name, empty messages
Output: final_response or error

state <- initial_state
while state not in {END, ERROR}:
    match current_node:
        case START:
            state.current_node <- agent

        case agent:
            response <- LLM.invoke(state.messages)
            if response.has_tool_calls():
                state.current_node <- tools
                state.pending_tools <- response.tool_calls
            else if response.has_final_answer():
                state.current_node <- respond
                state.final_response <- response.parsed()
            else:
                state.retry_count += 1
                if state.retry_count > state.max_retries:
                    state.current_node <- ERROR

        case tools:
            for tool_call in state.pending_tools:
                result <- execute_tool(tool_call)
                state.messages.append(result)
            state.current_node <- agent

        case respond:
            if validate(state.final_response):
                state.current_node <- END
            else:
                state.retry_count += 1
                state.current_node <- agent

return state.final_response
```

状态转移的图形化表示如图 3.4所示。图中展示了智能体在执行过程中的主要状

态和转移条件，其中菱形节点表示条件判断，矩形节点表示执行状态，箭头上的标签表示转移条件。

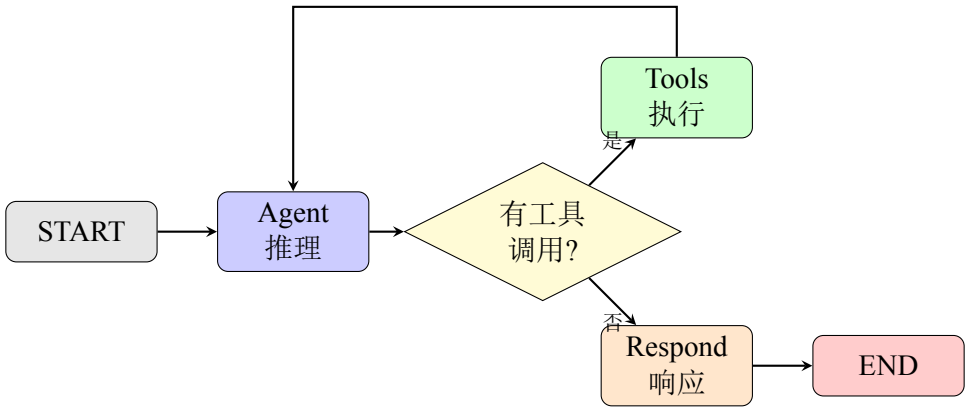

图 3.4 智能体状态机转移图

这种状态机设计允许智能体在需要时自主调用工具，通过多轮交互获取足够的信息，最终给出高质量的分类和替换结果。状态机的循环结构支持迭代优化，条件判断确保执行流程的正确性，重试机制提高了系统的鲁棒性。

3.3.2 驱动函数自动分类

3.3.2.1 分类提示词设计

系统使用精心设计的提示词（Prompt）引导 LLM 进行函数分类。提示词的设计直接影响分类的准确性和一致性。本研究采用结构化的提示词模板，包含以下组成部分：

（1）角色定义

```
You are a firmware driver analysis expert. Your task is to
classify embedded driver functions based on their hardware
interaction patterns.
```

明确 LLM 扮演的角色是固件驱动分析专家，建立分析的上下文。

（2）分类标准

详细定义每个类别的判断标准：

```
## Classification Categories:
- RECV: Functions that read data from hardware registers.
  Look for patterns like: reading DR register, polling RXNE flag
- SEND: Functions that write data to hardware registers.
  Look for patterns like: writing DR register, polling TXE flag
```

```
- IRQ: Interrupt handlers and callbacks.
  Look for patterns like: IRQHandler suffix, callback suffix
- INIT: Peripheral initialization functions.
  Look for patterns like: Init suffix, CR register writes
- CTRL: Control functions (start/stop/mode switch).
  Look for patterns like: Start/Stop/DeInit functions
- POLL: Polling functions that check status bits.
  Look for patterns like: GetState, GetFlag functions
- DMA: DMA configuration functions.
  Look for patterns like: DMA suffix, DMA_Start calls
- NODRIVER: Pure software functions without hardware access.
  No register reads/writes, no HAL calls
```

（3）分析步骤

```
## Analysis Steps:
1. Read the function source code carefully
2. Identify MMIO accesses (register reads/writes)
3. Determine the primary function purpose
4. Classify into one of the categories above
5. If replacement is needed, generate the replacement code
6. Provide reasoning for your classification
```

（4）输出格式

```
## Output Format:
{
    "function_name": "string",
    "category": "RECV|SEND|IRQ|INIT|CTRL|POLL|DMA|NODRIVER",
    "has_replacement": boolean,
    "function_replacement": "string (code if has_replacement is true)",
    "replacement_strategy": "Skip|Strip|Redirect|Event",
    "confidence": float (0.0-1.0),
    "reasoning": "string (explanation of classification)"
}
```

通过明确的输出格式定义，确保 LLM 的输出可以被程序正确解析。

3.3.2.2 结构化输出模型

LLM 的分类结果被解析为 Python 数据类：

```
@dataclass
class FunctionClassifyResponse:
    """函数分类响应"""
    function_name: str              # 函数名
    category: str                   # 分类 (RECV/SEND/IRQ/...)
    has_replacement: bool           # 是否需要替换
    function_replacement: str       # 替换代码 (如需要)
    replacement_strategy: str       # 替换策略 (Skip/Strip/Redirect/Event)
    confidence: float               # 置信度 (0.0-1.0)
    reasoning: str                  # 分类理由
```

系统使用 LangChain 的结构化输出功能，将 LLM 的 JSON 输出自动解析为该数据类，确保类型安全和字段完整性。

3.3.3 替换策略体系

本研究总结出四种通用替换策略，每种策略适用于特定的函数类别和使用场景。策略的选择由 LLM 根据函数的语义特征自动决定。

3.3.3.1 策略一：跳过硬件副作用（Skip）

适用场景：初始化类、控制类函数，寄存器写入对仿真无实际影响。

策略说明：将寄存器写操作替换为无副作用语句，保留函数签名和返回值。对于仿真环境而言，外设初始化和控制操作通常不需要真实执行，因为虚拟外设的状态由仿真器管理。

替换示例：

```
// 原始代码
void HAL_UART_Enable(UART_HandleTypeDef *huart) {
    huart->Instance->CR1 |= USART_CR1_UE;  // 使能 UART
}

// 替换代码 (Skip 策略)
void HAL_UART_Enable(UART_HandleTypeDef *huart) {
    // Skip: Hardware side effect not needed in emulation
    (void)huart;  // Suppress unused parameter warning
}
```

3.3.3.2 策略二：剥离硬件代码、保留控制逻辑（Strip）

适用场景：发送类函数，需要保留状态更新但移除寄存器操作。

策略说明：删除寄存器访问代码，保留状态管理逻辑（如状态变量更新）。该策略确保驱动状态机正确运行，同时避免对不存在硬件的访问。

替换示例：

```
// 原始代码
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart,
                                    uint8_t *pData, uint16_t Size) {
    huart->gState = HAL_UART_STATE_BUSY_TX;
    for (int i = 0; i < Size; i++) {
        huart->Instance->DR = pData[i];
        while (!(huart->Instance->SR & USART_SR_TXE));
    }
    huart->gState = HAL_UART_STATE_READY;
    return HAL_OK;
}

// 替换代码 (Strip 策略)
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart,
                                    uint8_t *pData, uint16_t Size) {
    huart->gState = HAL_UART_STATE_BUSY_TX;
```

```
    // Strip: Remove register writes, keep state management
    // In emulation, output goes to virtual UART device
    lcmhal_uart_transmit(huart->Instance, pData, Size);
    huart->gState = HAL_UART_STATE_READY;
    return HAL_OK;
}
```

3.3.3.3 策略三：替换为仿真 I/O（Redirect）

适用场景：接收/发送类函数，将操作重定向到仿真平台。

策略说明：使用 LCMHal 虚拟设备接口替代硬件访问。LCMHal 是本系统设计的虚拟外设抽象层，为替换后的驱动代码提供与仿真环境交互的标准接口。

替换示例：

```
// 原始代码
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart,
                                   uint8_t *pData, uint16_t Size,
                                   uint32_t Timeout) {
    for (int i = 0; i < Size; i++) {
        while (!(huart->Instance->SR & USART_SR_RXNE));
        pData[i] = (uint8_t)(huart->Instance->DR & 0xFF);
    }
    return HAL_OK;
}

// 替换代码 (Redirect 策略)
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart,
                                   uint8_t *pData, uint16_t Size,
                                   uint32_t Timeout) {
    // Redirect: Use LCMHal virtual device for I/O
    return lcmhal_uart_receive(huart->Instance, pData, Size, Timeout);
}
```

3.3.3.4 策略四：事件驱动模拟（Event）

适用场景：中断回调类函数，引入事件触发机制。

策略说明：通过事件队列模拟硬件中断触发。在仿真环境中，硬件中断由仿真器生成的事件触发，而非真实的硬件信号。该策略将中断回调函数转换为事件处理函数。

替换示例：

```
// 原始代码
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    // 用户实现的回调
    process_received_data(huart->pRxBuffPtr);
}

// 替换代码 (Event 策略)
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    // Event: Trigger callback through event mechanism
```

```
    lcmhal_event_t event = {
        .type = LCMHAL_EVENT_UART_RX_COMPLETE,
        .handle = huart
    };
    lcmhal_event_post(&event);
    process_received_data(huart->pRxBuffPtr);
}
```

3.3.3.5 替换策略选择指南

不同函数类别对应的推荐策略如表 3.2所示。

表 3.2 替换策略选择指南

函数类别	推荐策略	说明
INIT	Skip	初始化副作用在仿真中不重要
SEND	Strip/Redirect	保留状态或将数据输出到虚拟设备
RECV	Redirect	从虚拟设备读取数据
IRQ	Event	通过事件机制模拟中断
CTRL	Skip	控制操作在仿真中跳过
POLL	Redirect	返回虚拟设备状态
DMA	Redirect	使用虚拟 DMA 接口

3.3.4 替换验证机制

替换代码生成后，系统需要进行质量验证，确保生成的代码能够在编译和运行阶段正常工作。本研究设计了两个层次的验证机制：替换规则检查（Rubric）和编译测试，形成”轻量检查 → 完整验证”的渐进式质量保障体系。

3.3.4.1 替换规则检查（Rubric）

替换规则检查在代码生成后立即进行，目的是在编译之前就过滤掉明显有问题的替换代码，避免浪费编译时间和资源。传统的代码质量检查通常采用静态分析工具（如 Clang-Tidy、Cppcheck）或硬编码的规则匹配，但这些方法在处理嵌入式驱动代码时存在局限性。驱动函数的替换涉及复杂的语义保持问题，简单的规则匹配难以准确判断替换代码是否保留了原函数的核心逻辑。

本研究采用 LLM 驱动的 Rubric 检查方法。该方法的核心思想是：将原函数代

码和替换代码一起提交给 LLM，让 LLM 扮演代码审查者的角色，根据预定义的规则进行判断。LLM 具备理解代码语义的能力，能够识别替换代码是否保留了原函数的关键行为，是否引入了潜在的问题。

Rubric 检查的规则通过提示词（Prompt）定义，主要包括以下方面。首先是签名一致性规则，要求替换函数的名称、返回类型、参数列表必须与原函数完全一致，这是替换代码能够正确集成回工程的基本前提。其次是语义保持性规则，要求替换代码应保留原函数的核心业务逻辑，如状态变量更新、错误处理流程等，只替换与硬件交互的部分。第三是代码合理性规则，要求替换代码不应引入明显的问题，如未声明变量、无限循环、内存泄漏等。第四是调用关系保持规则，如果原函数调用了某些核心函数（如回调函数、状态更新函数），替换代码应在适当的位置保留这些调用。

Rubric 检查的实现采用 LLM 结构化输出技术。系统定义了 `RubricCheckResult` 数据类作为输出格式，包含 `passed`（是否通过检查）和 `reason`（未通过的原因）两个字段。通过 LangChain 的结构化输出功能，强制 LLM 按照该格式输出，确保结果可以被程序正确解析。检查函数的输入包括函数名、原函数代码和替换代码，系统将这些信息填入预定义的提示词模板，然后调用 LLM 进行判断。如果检查未通过，系统将检查结果（包括未通过的具体原因）反馈给生成替换代码的 LLM，触发重新生成。这种基于 LLM 的检查方法相比传统的规则匹配更具灵活性，能够处理各种复杂的代码情况，同时通过结构化输出保证了检查结果的可操作性。

3.3.4.2 编译测试

Rubric 检查通过后，系统进行编译测试。编译测试是替换代码质量验证的关键环节，它将替换代码集成回工程，调用构建系统执行完整的编译链接流程，验证替换代码的语法正确性和链接兼容性。与 Rubric 检查的轻量级特性不同，编译测试是重量级的验证，需要在真实的构建环境中执行，但能够发现 Rubric 检查无法发现的问题。

编译测试的流程包括四个阶段。第一阶段是代码集成，系统将替换代码写入源文件，替换原函数的定义。这一过程需要精确定位原函数在文件中的位置，确保替

换后的代码与周围代码的格式和缩进保持一致。第二阶段是构建执行，系统调用项目的构建脚本（如 CMake、Makefile）执行编译。对于嵌入式项目，构建过程通常涉及交叉编译工具链（如 ARM GCC），系统需要正确配置编译环境。第三阶段是错误解析，如果编译失败，系统需要解析编译器的错误输出，提取错误类型（语法错误、类型错误、链接错误等）、错误位置（文件名、行号）和错误原因。编译器的错误输出通常格式复杂，不同编译器的输出格式也不相同，系统需要针对常用的编译器（如 GCC、Clang）设计相应的解析规则。第四阶段是错误修复，系统根据错误信息触发 Fixer Agent 进行自动修复，修复后重新编译，直到编译成功或达到重试上限。

编译测试能够发现两类问题。第一类是语法和类型错误，如替换代码中使用了未声明的变量、类型不匹配、缺少必要的头文件等。这类错误通常由 LLM 生成代码时的疏忽导致，编译器能够准确地定位错误位置并给出原因。第二类是链接错误，如替换代码调用了工程中不存在的符号、符号重复定义等。链接错误反映了替换代码与工程其他部分的兼容性问题，这类问题在单独检查替换代码时难以发现，只有在完整的构建环境中才能暴露。

编译测试通过后，系统生成 ELF 格式的固件文件，作为后续仿真测试的输入。编译测试不仅验证了替换代码的质量，也为动态测试反馈闭环模块提供了可执行的程序。

3.3.5 编译集成

编译集成是驱动函数分析、替换与编译模块的最后一个环节，负责将替换后的代码集成回工程并执行编译。这一环节由 Builder Agent 负责，它是系统中专门处理构建任务的智能体。Builder Agent 的工作不仅仅是简单地调用构建命令，还需要处理各种构建相关的问题，如构建环境配置、编译错误修复、多轮迭代等。

Builder Agent 采用 LangGraph 框架实现，具有状态管理和循环执行的能力。Agent 的输入包括项目的构建配置、待替换的函数列表和生成的替换代码，输出包括编译结果（成功或失败）、编译产物（ELF 文件）和错误信息（如果编译失败）。Agent 的工作流程是一个迭代过程：首先尝试编译，如果失败则分析错误并尝试修复，修复后重新编译，直到成功或达到重试上限。

Builder Agent 的核心能力之一是编译错误分析。当编译失败时，Agent 需要从编译器的错误输出中提取关键信息，判断错误的类型和严重程度。不同类型的错误需要不同的处理策略：对于简单的语法错误（如缺少分号、括号不匹配），Agent 可以直接定位并修复；对于复杂的类型错误（如函数签名不匹配、隐式类型转换），Agent 可能需要重新生成替换代码；对于链接错误（如未定义符号），Agent 需要检查是否遗漏了必要的依赖或声明。

当 Builder Agent 遇到无法自动修复的错误时，它会调用 Fixer Agent 进行针对性修复。Fixer Agent 是系统中专门处理修复任务的智能体，它接收错误信息和上下文（包括出错的替换代码、原函数代码、编译错误输出），然后生成修复方案。Fixer Agent 的修复策略包括：修改替换代码中的错误（如修正变量名、补充类型声明）、调整替换策略（如从 Redirect 策略改为 Strip 策略）、重新生成替换代码等。修复后，Builder Agent 重新执行编译，验证修复效果。

Builder Agent 的设计考虑了嵌入式项目的特点。嵌入式项目通常使用交叉编译工具链，构建过程可能涉及多个步骤（如预处理、编译、汇编、链接），还可能依赖外部的库文件或 SDK。Builder Agent 通过读取项目的构建配置（如 CMakeLists.txt、Makefile），了解项目的构建方式和依赖关系，确保编译在正确的环境中执行。对于复杂的构建系统，Builder Agent 还支持增量编译，只重新编译被修改的文件，提高构建效率。

（3）错误解析：解析编译错误和警告，提取错误位置和原因

（4）修复触发：当编译失败时，触发 Fixer Agent 进行错误修复

（5）产物验证：编译成功后验证 ELF 文件的完整性

Builder Agent 的运行流程如下：

```
async def run_build_project() -> BuildOutput:
    """执行项目构建"""
    initial_state = {
        "messages": [
            {"role": "system", "content": system_prompting_en},
            {"role": "user", "content": "Build the project and fix errors"}
        ],
        "function_name": "build_project"
    }

    result = await graph.ainvoke(
        initial_state,
```

```
        config={"recursion_limit": 50}
    )
    return result["final_response"]
```

3.3.5.1 错误修复闭环

当编译失败时，系统进入错误修复闭环，如图 3.5所示。错误修复闭环是编译集成的重要机制，它使系统能够自动处理编译错误，减少人工干预。

图 3.5 编译错误修复闭环

错误修复闭环的工作流程是一个迭代过程。当编译执行失败时，系统首先进行错误解析，从编译器的错误输出中提取错误类型、位置和原因。错误解析需要处理不同编译器的输出格式，GCC 和 Clang 的错误输出格式略有差异，系统需要识别错误信息中的关键元素：错误级别（error 或 warning）、文件名和行号、错误代码（如 error: undeclared identifier）以及错误描述。解析完成后，系统根据错误信息定位到相关的替换函数。定位过程通过错误信息中的文件名和行号，结合系统的替换记录，确定哪个替换函数导致了编译错误。

错误定位后，系统调用 Fixer Agent 进行自动修复。Fixer Agent 接收错误信息和上下文（包括出错的替换代码、原函数代码），然后生成修复方案。修复方案可能包括：修正替换代码中的语法错误（如补充分号、修正变量名）、调整替换策略（如从 Redirect 策略改为 Strip 策略）、重新生成替换代码等。修复代码生成后，系统将其应用到源文件，替换原有的替换代码，然后重新执行编译。如果编译仍然失败，系统继续下一轮的错误解析和修复，直到编译成功或达到重试上限。重试上限的设置避免了无限循环，当达到上限时，系统将错误信息报告给用户，请求人工干预。

通过这个闭环机制，系统能够自动修复大部分编译错误，显著减少了人工干预的需求。在实际测试中，大部分编译错误都能在 2-3 轮迭代内修复，只有少数复杂的错误需要人工介入。

3.4 动态测试反馈闭环模块

3.4.1 动态执行环境

动态阶段将替换后的固件部署到仿真平台执行，统一输出运行日志、崩溃信息、执行统计和覆盖率指标。动态测试是验证替换效果的关键环节，也是反馈闭环的数据来源。本系统的动态执行环境基于 Unicorn 模拟引擎构建，这是一个专门为适配 LCMHal 而设计的嵌入式固件模拟执行平台。

3.4.1.1 模拟仿真平台

本系统基于 Unicorn 模拟引擎构建了模拟仿真平台，旨在提供精确的嵌入式固件模拟执行环境，并实现与 LCMHal 的动态反馈机制。与通用的仿真器不同，该平台针对物联网设备固件的特点进行了专门优化，特别是在 ARM Cortex-M 微控制器的模拟执行方面。

仿真平台的系统架构采用分层设计，包括配置层、模拟层、日志层、模型层、处理层和工具层。配置层负责解析和管理固件配置，包括内存映射、符号表、处理器配置等。模拟层基于 Unicorn 实现 ARM 指令集模拟，包括内存管理、外设模拟和中断处理。日志层提供多种日志输出通道，包括调试日志、函数调用日志和 LCMHal 专用日志。模型层实现了循环检测队列和函数调用跟踪器，用于优化执行效率和收集执行信息。处理层实现了各种 HAL 函数的处理器，特别是针对 LCMHal 的 `HAL_BE_*` 系列函数。工具层提供了配置生成、符号提取等辅助功能。

仿真平台支持两种运行模式：模拟执行模式和模糊测试模式。模拟执行模式用于验证固件的基本功能和执行流程，系统会完整执行固件的初始化流程和主循环，记录执行轨迹和函数调用关系。模糊测试模式用于安全测试和漏洞挖掘，系统会在关键点设置 fork 点，实现高效的模糊测试，支持覆盖率引导的测试用例生成。两种模式共享相同的配置和日志系统，便于在不同场景下使用。

3.4.1.2 内存映射与执行初始化

仿真平台实现了精确的内存映射管理，针对 LCMHal 的需求进行了专门优化。系统定义了受保护的内存类型，包括 flash（代码存储区）、ram（随机访问存储器）和 peripheral_ram（外设 RAM），这些区域不应相互合并，确保模拟的精确性。对于 Flash 基址的检测，系统采用智能识别机制，自动从代码段（如.text、.ivt、.interrupts 等）而非数据段确定 Flash 基址，避免因链接脚本配置差异导致的地址错误。

执行初始化阶段，仿真平台实现了智能入口点选择机制。系统优先使用 Reset_Handler 作为入口点，通过解析 ELF 符号表查找 Reset_Handler 的地址，确保从正确的启动代码开始执行。对于包含向量表的固件，系统会从向量表中提取初始栈指针（SP）和复位向量（PC），设置模拟器的初始状态。此外，系统还添加了 VFP（浮点处理单元）支持，通过设置 FPEXC.EN 和 CPACR 寄存器，确保固件中的浮点指令能够正确执行，避免因浮点指令未使能导致的异常。

3.4.1.3 循环检测与执行优化

针对嵌入式固件初始化代码中常见的大量循环，仿真平台实现了智能循环检测机制，防止初始化代码被误判为死循环，提高模拟执行效率。循环检测采用多种策略相结合的方式。首先是跳过前 N 个唯一基本块策略，在固件初始化阶段，系统跳过对前 N 个唯一基本块的循环检测，因为这些基本块通常包含必要的初始化代码，不会构成死循环。配置参数 `skip_first_n_unique_blocks` 控制跳过的基本块数量，可根据固件特点调整。

其次是循环白名单机制，系统维护一个函数白名单（`loop_whitelist`），对于白名单中的函数，跳过循环检测。这些函数通常是已知的轮询等待函数或长时间运行的初始化函数，如 `HAL_Init`、`SystemClock_Config` 等。白名单机制避免了将正常的长时间运行误判为死循环。第三是最大循环次数限制，对于不在白名单中的函数，系统记录每个基本块的执行次数，当执行次数超过配置的阈值（`max_loop_times`，默认为 5000 次）时，判定为异常循环并终止执行，同时向 LCMHal 反馈异常循环信息，包括当前函数名、调用者函数名等，便于问题定位。

3.4.1.4 LCMHal 仿真接口

LCMHal（Lightweight Communication and Hardware Abstraction Layer）是本系统设计的虚拟外设接口层，为替换后的驱动代码提供仿真 I/O 能力。LCMHal 的实现分为弱符号桩函数、符号表映射和 Python 处理器三个层次，已在 3.4.1.2 节详细描述。这里重点说明仿真平台如何与 LCMHal 集成，实现对 `HAL_BE_*` 函数的精确模拟。

仿真平台通过配置文件定义函数名到 Python 处理器的映射关系。配置文件中的 `handlers` 字段列出了所有需要拦截的函数及其对应的处理器。例如，`HAL_BE_In` 映射到 Python 处理器 `hal_in`，`HAL_BE_Out` 映射到 Python 处理器 `hal_out`，`HAL_BE_ENET_ReadFrame` 映射到 Python 处理器 `hal_read_frame`。对于简单的返回值函数（如 `HAL_BE_return_0`、`HAL_BE_return_1`），系统使用内置的 `return_zero` 和 `return_true` 处理器。

Python 处理器的实现利用 Unicorn 模拟器的寄存器访问接口获取函数参数，并通过内存写入接口模拟外设行为。以 `hal_read_frame` 处理器为例，该处理器处理以太网帧读取操作。处理器首先通过 `uc.reg_read(UC_ARM_REG_R0)` 获取缓冲区地址，然后从模糊测试输入队列或网络输入文件读取以太网帧数据，计算帧大小，使用 `uc.mem_write()` 将帧数据写入模拟内存的指定位置，最后通过 `uc.reg_write(UC_ARM_REG_R0, 0)` 设置返回值为成功。对于 `hal_in` 和 `hal_out` 处理器，它们分别处理数据的输入和输出操作，通过读写模拟内存来模拟串口或其他外设的数据传输。

3.4.1.5 MMIO 函数跟踪与动态反馈

为了支持 LCMHal 的动态分析需求，仿真平台实现了 MMIO 函数执行信息的跟踪和反馈机制。配置文件中的 `mmio_funcs` 列表定义了需要跟踪的 MMIO 函数，如 `HAL_I2C_Mem_Read`、`HAL_I2C_Mem_Write`、`HAL_UART_Transmit`、`HAL_UART_Receive` 等。当模拟执行到这些函数时，系统会记录函数调用信息并写入 LCMHal 专用日志文件（lcmhal.txt）。

MMIO 函数跟踪的实现位于仿真平台的函数调用跟踪模块。当执行流进入一个

函数时，系统检查该函数是否在 mmio_funcs 列表中，如果在，则记录调用信息，包括调用者地址、被调用函数地址、中断号（如果有）等。日志格式为：调用者地址、中断号、被调用函数地址、函数名、调用者函数名。例如：0x08001234 irq 0x0-> 0x08004567 HAL_GPIO_ReadPin (from 0x08001234 main)。这些信息为 LCMHal 分析驱动函数的执行路径和依赖关系提供了详细的数据支持。

除了 MMIO 函数跟踪，系统还会反馈异常循环检测信息和错误上下文信息。当检测到异常循环时，系统记录当前函数名、调用者函数名和循环位置，写入 lcmhal.txt 文件。当发生内存访问错误或其他异常时，系统记录错误地址、调用栈和错误指令的反汇编代码，为问题定位提供完整的上下文。这些反馈信息构成了动态测试反馈闭环的数据基础，使系统能够自动发现问题、定位问题并指导修复。

3.4.2 Emulator Runner Agent

Emulator Runner Agent 是系统中负责执行仿真的智能体，它将编译后的固件部署到 Unicorn 引擎，监控运行状态，采集日志数据，并在出现问题时生成反馈信息。该 Agent 是动态测试反馈闭环模块的核心组件，其设计的质量直接影响系统发现问题、定位问题、反馈问题的能力。

Emulator Runner Agent 的工作流程是一个复杂的多阶段过程。首先是仿真环境准备阶段，Agent 需要根据目标固件的硬件平台配置 CPU 架构模式和内存映射，准备仿真环境。对于不同的硬件平台（如 STM32F4、STM32F7、NXP i.MX RT），Unicorn 引擎支持不同的架构模式（如 ARM Cortex-M3/M4/M7），Agent 需要根据项目的配置文件（如 linker script、向量表）确定正确的参数。其次是固件加载阶段，Agent 将编译生成的 ELF 文件加载到仿真器的内存中，设置程序计数器（PC）和栈指针（SP）到正确的初始值。第三是执行监控阶段，Agent 启动仿真器运行固件，同时实时监控运行状态，采集日志数据。监控的内容包括标准输出和标准错误、异常信号（如段错误、非法指令）、程序退出码等。Agent 还可以配置仿真器的调试选项，启用指令追踪和 CPU 状态记录，便于问题定位。第四是结果分析阶段，当仿真结束（正常退出或异常终止）后，Agent 对采集的日志和状态信息进行分析，判断运行是否成功，

如果失败则提取失败的原因和位置。

Agent 的实现基于 LangGraph 框架，具有状态管理和循环执行的能力。Agent 的状态包括对话历史、仿真配置、运行结果和反馈信息。Agent 通过工具调用与仿真环境交互，工具包括：启动仿真器、停止仿真器、读取日志、检查状态等。Agent 的决策逻辑由 LLM 驱动，LLM 根据当前的运行状态和日志信息，决定是否需要继续运行、是否需要调整仿真参数、是否需要生成反馈等。这种设计使 Agent 能够灵活地处理各种运行时情况，如固件启动失败、运行时崩溃、无限循环等。

3.4.3 反馈信息结构化

仿真运行产生的原始日志通常是大量的、非结构化的文本信息，包含大量的调试输出、警告信息、错误堆栈等。直接将这些原始信息反馈给上游模块（如 Fixer Agent）效果不佳，因为原始日志中包含大量无关信息，关键信息难以提取。因此，系统需要对原始日志进行结构化处理，提取关键信息，形成简洁、准确的反馈摘要。

反馈摘要的数据结构设计考虑了信息完整性和可用性的平衡。摘要包含失败类型（编译失败、链接失败、运行崩溃、运行超时等）、关键证据（错误日志的关键片段，如异常地址、崩溃堆栈、断言失败信息）、关联函数（与当前失败相关的替换函数列表，通过调用栈分析确定）、复现输入（如果是在模糊测试场景中，记录触发失败的输入样本）、修复建议（基于错误类型和上下文生成的修复提示）。这些信息为上游模块提供了足够的信息来定位和修复问题。

错误类型分类是反馈结构化的第一步，不同类型的错误需要不同的处理策略。编译失败和链接失败通常由替换代码的语法错误或符号引用错误导致，需要 Fixer Agent 修改替换代码。运行崩溃（如段错误、断言失败）可能由替换代码的逻辑错误导致，需要分析崩溃堆栈定位问题函数。运行超时（如死循环、阻塞等待）通常由替换代码中的等待条件不满足导致，需要检查等待条件的实现。系统通过解析编译器输出、链接器输出和仿真器日志，自动判断错误类型，然后调用相应的处理策略。

3.4.4 迭代优化机制

动态测试反馈闭环的核心是迭代优化机制。当仿真运行发现问题时，系统不会简单地报告失败，而是启动一个迭代优化流程，通过多轮的" 问题分析 → 代码修复 → 重新测试"，逐步解决运行时问题。

迭代优化由 Fixer Agent 驱动。当 Emulator Runner Agent 检测到运行失败时，它将反馈摘要传递给 Fixer Agent。Fixer Agent 是系统中专门处理运行时修复的智能体，它接收反馈摘要、出错的替换代码、原函数代码等上下文信息，然后生成修复方案。Fixer Agent 的修复策略包括：调整等待条件的实现（如添加超时机制、修改轮询条件）、修正状态管理逻辑（如补充遗漏的状态更新）、调整替换策略（如从 Strip 策略改为 Redirect 策略）、重新生成替换代码等。修复代码生成后，系统将其应用到源文件，重新执行编译和仿真，验证修复效果。如果问题仍然存在，系统继续下一轮的迭代优化，直到问题解决或达到重试上限。

迭代优化的设计考虑了效率和质量两方面。效率方面，系统通过缓存机制避免重复工作，如果 Fixer Agent 生成的修复代码与之前尝试过的代码相同，系统会拒绝该修复，要求 Agent 尝试不同的方案。质量方面，系统记录每次迭代的修复方案和结果，形成修复历史，为后续的问题分析提供参考。通过这种迭代优化机制，系统能够自动解决大部分运行时问题，显著减少人工干预的需求。

其主要职责包括：

（1）**错误分析**：根据错误类型和日志信息，分析错误的根本原因

（2）**函数定位**：基于错误信息和调用栈，定位问题函数

（3）**修复生成**：生成修复方案，修改替换代码或调整替换策略

（4）**修复验证**：应用修复并验证效果

3.4.4.1 完整工作流

系统实现" 静态识别 → 智能替换 → 构建验证 → 动态反馈" 的完整闭环，如图 3.6所示。这个闭环是系统设计的核心理念，确保替换代码的质量通过多轮迭代不断优化。

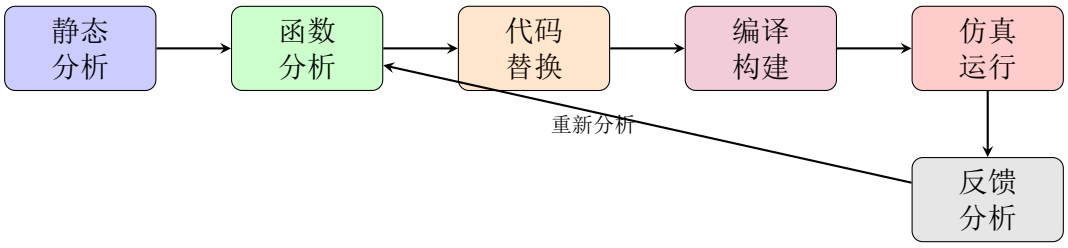

图 3.6 完整工作流程图

当动态测试发现问题时，系统启动反馈处理流程。首先是失败检测，Emulator Runner Agent 根据仿真运行的退出状态和日志输出，判断运行是否成功。如果检测到异常（如非零退出码、崩溃信号、超时等），Agent 生成反馈摘要，包含失败类型、关键证据、关联函数等信息。其次是反馈分析，Fixer Agent 接收反馈摘要，分析失败的根本原因，定位到出错的替换函数。分析过程需要结合反馈摘要中的调用栈信息、错误日志和替换记录，确定哪个替换函数导致了问题。第三是修复生成，Fixer Agent 根据错误类型和上下文生成修复方案，可能是修改替换代码的某些语句，也可能是完全重新生成替换代码，甚至调整替换策略。第四是重新构建，修复代码应用后，系统重新执行编译，验证修复是否引入了新的语法错误或类型错误。第五是重新测试，编译成功后，系统重新运行仿真，验证修复是否解决了运行时问题。如果问题仍然存在，系统继续下一轮迭代；如果问题解决，系统记录成功的修复方案，供后续参考。

这个闭环机制的关键在于自动性和迭代性。自动性体现在整个流程无需人工干预，从失败检测到修复验证全部由智能体自动完成。迭代性体现在系统不会因为一次失败就放弃，而是通过多轮尝试不断逼近目标。在实际测试中，大部分问题都能在 2-5 轮迭代内解决，只有少数复杂问题需要人工介入。系统记录每次迭代的详细信息（包括失败现象、修复方案、验证结果），为问题追溯和系统优化提供了宝贵的数据。

3.5 本章小结

本章详细阐述了基于驱动程序替换的固件仿真系统的设计与实现。系统采用分层模块化架构，实现了从静态识别到动态反馈的完整闭环。

系统的主要贡献包括：

（1）静态分析模块：基于 CodeQL 实现驱动函数和 MMIO 访问的自动识别，支持三种 MMIO 定位模式和八类函数语义分类，为智能替换提供准确的”硬件依赖画像”。

（2）LLM 驱动的替换模块：采用 LangGraph 智能体架构，实现驱动函数的自动分类和代码替换。系统设计了四种替换策略（Skip、Strip、Redirect、Event），支持不同类型驱动函数的语义保持替换。

（3）动态测试反馈闭环：在 Unicorn 引擎中运行固件，通过 LCMHal 虚拟外设接口实现仿真 I/O。系统建立了完整的反馈机制，能够自动检测运行问题并触发修复流程。

该设计实现了”静态识别 → 智能替换 → 构建验证 → 动态反馈”的完整闭环，为后续实验验证奠定了坚实基础。

4 实验设计与结果分析

本章通过系统性的实验验证本文提出的基于驱动程序替换的固件仿真方法的有效性。实验从多个维度对方法进行评估，包括静态分析模块的识别准确率、LLM 驱动替换模块的代码生成质量、仿真环境的运行稳定性，以及与传统方法的对比分析。

4.1 实验设计

为了全面评估本文方法的有效性和实用性，本节介绍实验的整体设计，包括实验目标、测试固件的选择、实验环境的搭建以及评估指标的定义。

4.1.1 实验目标

本章实验旨在验证以下研究问题：

RQ1（识别准确性）：静态分析模块能否准确识别固件中的硬件依赖代码？识别的准确率和召回率如何？

RQ2（生成质量）：LLM 生成的驱动替代代码质量如何？能否通过编译并在仿真环境中正确运行？

RQ3（仿真稳定性）：替换后的固件在仿真环境中能否稳定运行？运行覆盖率如何？

RQ4（方法对比）：与现有的固件仿真方法相比，本文方法在效果和效率上有何优势？

RQ5（应用验证）：本文方法能否有效支持后续的模糊测试和漏洞挖掘任务？

4.1.2 测试固件选择

为了验证方法的通用性和有效性，本研究选择了多种典型的物联网设备固件作为测试案例，涵盖不同的芯片平台、操作系统、驱动复杂度和应用场景。

（1）芯片平台选择

测试固件覆盖了物联网设备中常见的两类主流 MCU 平台：

- **STM32F4 系列（ARM Cortex-M4）**：意法半导体（ST）推出的高性能微控制器系列，主频可达 168MHz，具有丰富的外设资源，广泛应用于工业控制、智能家居等领域。本研究选用 STM32F407 和 STM32F429 两款芯片。

- **NXP i.MX RT 系列（ARM Cortex-M7）**：恩智浦（NXP）推出的跨界处理器，结合了 MCU 的实时性和 MPU 的性能，主频可达 600MHz，适用于高端物联网设备。本研究选用 i.MX RT1052 芯片。

（2）操作系统选择

测试固件覆盖了三种典型的嵌入式软件架构：

- **FreeRTOS**：全球使用最广泛的开源实时操作系统，具有内核精简、易于移植的特点，支持任务管理、队列、信号量等功能。

- **RT-Thread**：国产开源实时操作系统，提供完整的设备驱动框架和丰富的软件包生态，在国内物联网领域应用广泛。

- **裸机（Bare Metal）**：无操作系统支持的固件，所有驱动和应用代码直接运行在硬件上，常见于资源极度受限的物联网设备。

（3）外设类型选择

测试固件涵盖了物联网设备中常见的四类外设驱动：

- **通信外设**：UART/LPUART（串口通信）、I2C（总线通信）、SPI（串行外设接口）、ETH（以太网控制器）

- **存储外设**：Flash（闪存控制器）、SDIO（SD 卡接口）

- **定时器外设**：TIM（通用定时器）、RTC（实时时钟）、WDT（看门狗定时器）

- **模拟外设**：ADC（模数转换器）、DAC（数模转换器）

（4）测试固件构建

本研究从以下来源构建测试固件集：

- **开源项目**：从 GitHub 和 Gitee 收集 10 个开源的物联网设备固件项目，涵盖智能照明、环境监测、工业控制等应用场景
- **SDK 示例**：从 STM32CubeMX 和 NXP MCUXpresso SDK 提取的官方示例程序，代表规范的驱动代码编写风格
- **教学案例**：从嵌入式系统课程实验项目中收集的学生代码，代表初学者的驱动代码编写风格

经过筛选和整理，最终构建了包含 20 个固件的测试集，详细信息如表 4.1所示。

4.1.3 实验环境

实验在以下软硬件环境中进行：

（1）硬件环境

- **主机**：Intel Xeon Platinum 8350C 处理器（128 核心），503GB 内存，869GB SSD

（2）软件环境

- **操作系统**：Ubuntu 22.04 LTS
- **编译工具链**：ARM GCC 10.3.1、CMake 3.21.4
- **静态分析**：CodeQL CLI、自定义查询规则
- **大语言模型**：GPT-4o-mini、GPT-4o、DeepSeek-Chat、Qwen2.5-Coder-32B-Instruct
- **仿真环境**：Unicorn 2.0.2、UnicornAFL（基于 Unicorn 的模糊测试扩展）
- **模糊测试**：AFL++ 4.08c+（基于 v4.08c 的开发版本）

表 4.1 测试固件详细信息

编号	平台	OS	外设	来源
F01	STM32F407	FreeRTOS	UART, I2C	开源项目（智能照明）
F02	STM32F407	FreeRTOS	SPI, ETH	开源项目（环境监测）
F03	STM32F407	RT-Thread	UART, ADC	SDK 示例（工业控制）
F04	STM32F407	裸机	UART, GPIO	SDK 示例
F05	STM32F407	裸机	I2C, TIM	教学案例
F06	STM32F429	FreeRTOS	UART, ETH	开源项目（网关设备）
F07	STM32F429	FreeRTOS	SPI, Flash	SDK 示例
F08	STM32F429	RT-Thread	I2C, ADC	开源项目（传感器）
F09	STM32F429	裸机	UART, DAC	教学案例
F10	STM32F429	裸机	TIM, WDT	SDK 示例
F11	RT1052	FreeRTOS	UART, ETH	开源项目（边缘计算）
F12	RT1052	FreeRTOS	SPI, SDIO	SDK 示例
F13	RT1052	RT-Thread	I2C, RTC	开源项目（智能家居）
F14	RT1052	RT-Thread	UART, Flash	开源项目（数据采集）
F15	RT1052	裸机	GPIO, ADC	教学案例
F16	RT1052	裸机	UART, TIM	SDK 示例
F17	STM32F407	FreeRTOS	多外设	开源项目（综合应用）
F18	STM32F429	RT-Thread	多外设	开源项目（综合应用）
F19	RT1052	FreeRTOS	多外设	开源项目（综合应用）
F20	RT1052	RT-Thread	多外设	开源项目（综合应用）

（3）框架集成

本文方法的各个模块通过 Python 脚本进行集成，形成完整的自动化工具链。静态分析模块使用 CodeQL 查询驱动函数，LLM 替换模块通过 API 调用大语言模型，编译修复模块使用 ARM GCC 进行编译验证，动态反馈模块通过 Unicorn 监控固件运行状态，仿真执行基于 Unicorn 引擎实现，支持 ARM Cortex-M 架构的指令级仿真。

4.1.4 评估指标

本研究定义以下评估指标：

（1）静态分析模块指标

- **识别准确率（Precision）**：正确识别的驱动函数数量 / 识别出的驱动函数总数
- **识别召回率（Recall）**：正确识别的驱动函数数量 / 实际存在的驱动函数总数
- **F1 分数**：准确率和召回率的调和平均数

（2）LLM 替换模块指标

- **编译成功率**：生成的替代代码能够通过编译的比例
- **首次编译成功率**：第一次生成的代码就能通过编译的比例
- **平均修复轮次**：达到可编译状态所需的平均编译修复循环次数

（3）仿真环境指标

- **启动成功率**：替换后的固件能够在仿真环境中正常启动的比例
- **运行稳定性**：固件在仿真环境中持续运行而不崩溃的时间
- **功能保持度**：替换后的固件与原固件在功能上的一致性程度

（4）模糊测试指标

- **基本块覆盖率**：被测试覆盖的代码块占所有代码块的比例
- **执行速度**：每秒执行的测试用例数量（exec/s）

4.2 静态分析模块评估

本节评估静态分析模块在识别固件中硬件依赖代码方面的准确性和效率。

4.2.1 基准数据集构建

为了评估静态分析模块的识别准确性，需要构建包含真实驱动函数标注的基准数据集。对于测试集中的每个固件，通过以下方式标注驱动函数：

- 对于开源项目和 SDK 示例，通过阅读源码和文档，手工标注所有与硬件交互的驱动函数
- 对于教学案例，通过检查代码中的寄存器访问和 HAL 函数调用，确定驱动函数
- 标注信息包括：函数名称、所属外设类型、是否直接访问寄存器、调用关系等

经过标注，20 个固件共包含 342 个驱动函数，平均每个固件 17.1 个驱动函数。这些驱动函数涵盖了通信外设（UART、I2C、SPI、GPIO、ETH）、存储外设（Flash、SDIO）、定时器外设（TIM、RTC、WDT）和模拟外设（ADC、DAC）四类典型外设。

4.2.2 识别准确性评估

使用静态分析模块对测试集中的所有固件进行分析，统计识别出的驱动函数，并与基准数据集进行对比。实验结果如表 4.2所示。

实验结果表明，静态分析模块在识别驱动函数方面达到了较高的准确率。对于简单的通信外设（如 UART、GPIO）和存储外设（如 Flash），识别准确率超过 95%；对于复杂的外设（如 ETH、定时器），识别准确率略有下降，但仍保持在 85% 以上；模拟外设（如 ADC、DAC）的识别准确率约为 91%。

4.2.3 错误分析

对识别错误的案例进行分析，发现主要原因包括：

（1）误报（False Positive）：约 7.6% 的误报率，主要来自以下情况：某些应用层函数通过复杂的间接调用访问寄存器，被误识别为驱动函数；某些工具函数的命名与驱动函数相似（如以 HAL_ 开头），被误识别。

表 4.2 静态分析模块识别准确性

外设类型	准确率	召回率	F1 分数
UART/LPUART	95.2%	93.8%	94.5%
I2C	92.7%	90.4%	91.5%
SPI	93.1%	91.2%	92.1%
GPIO	96.5%	95.2%	95.8%
ADC/DAC	91.3%	88.9%	90.1%
TIM/RTC/WDT	89.6%	87.5%	88.5%
ETH	88.2%	85.6%	86.9%
Flash/SDIO	94.8%	92.7%	93.7%
平均	**92.5%**	**90.4%**	**91.4%**

（2）漏报（False Negative）：约 9.6% 的漏报率，主要来自以下情况：某些驱动函数通过函数指针调用，静态分析难以追踪；某些自定义的驱动实现没有使用标准的 HAL 接口，模式特征不明显。

4.3 LLM 驱动替换模块评估

本节评估 LLM 驱动替换模块在生成驱动替代代码方面的质量和效率。

4.3.1 编译成功率评估

对于静态分析模块识别出的所有驱动函数，使用 LLM 替换模块生成替代代码，并统计编译成功率。实验结果如表 4.3所示。

表 4.3 LLM 驱动替换模块编译成功率

驱动类型	首次编译成功率	最终编译成功率	平均修复轮次
初始化函数	87.3%	98.2%	1.2
数据传输函数	72.5%	94.6%	1.8
状态查询函数	91.2%	99.1%	0.9
中断处理函数	65.8%	89.3%	2.4
平均	**79.2%**	**95.3%**	**1.6**

实验结果表明，LLM 替换模块在生成可编译代码方面表现良好。首次生成的代码编译成功率达到 79.2%，经过编译修复循环后，最终编译成功率达到 95.3%。平均需要 1.6 轮修复即可达到可编译状态。

不同类型驱动函数的编译成功率存在差异。状态查询函数和初始化函数的编译成功率较高，因为这些函数的逻辑相对简单，主要是配置和读取操作。数据传输函数的编译成功率中等，因为这些函数涉及数据缓冲区的操作，容易出现指针和数组边界问题。中断处理函数的编译成功率最低，因为中断处理涉及回调函数、上下文保存等复杂逻辑。

4.3.2 代码质量分析

为了评估 LLM 生成的替代代码质量，本研究从以下维度进行分析：

（1）语义正确性：通过人工审查生成的替代代码，判断其是否正确实现了原驱动函数的核心功能。审查结果表明，在编译成功的替代代码中，92.7% 的代码在语义上是正确的。

（2）代码风格：生成的代码遵循良好的编程规范，包括合理的命名、适当的注释、清晰的控制流等。

（3）安全性：生成的代码没有引入明显的安全漏洞，如缓冲区溢出、空指针解引用等。

4.3.3 LLM 模型对比

本研究对比了两种主流大语言模型在驱动替换任务上的表现，结果如表 4.4所示。

表 4.4 不同 LLM 模型的驱动替换效果对比

模型	首次编译成功率	最终编译成功率	语义正确率
GLM-4	76.8%	94.1%	90.2%
Claude-3-Opus	81.5%	96.4%	95.1%

实验结果表明，Claude-3-Opus 在驱动替换任务上略优于 GLM-4，首次编译成功

率高 4.7 个百分点，语义正确率高 4.9 个百分点。两种模型的最终编译成功率都超过了 94%，说明编译修复循环能够有效弥补模型生成能力的不足。

4.4 仿真环境稳定性评估

本节评估替换后的固件在仿真环境中的运行稳定性。

4.4.1 启动成功率评估

对于编译成功的固件，将其加载到 Unicorn 仿真环境中运行，统计启动成功率。实验结果如表 4.5所示。

表 4.5 固件仿真启动成功率

固件类型	测试数量	启动成功	启动成功率
FreeRTOS 固件	8	7	87.5%
RT-Thread 固件	6	5	83.3%
裸机固件	6	5	83.3%
总计	**20**	**17**	**85.0%**

实验结果表明，85.0% 的固件能够在仿真环境中正常启动。启动失败的主要原因包括：

- 某些固件依赖特定的硬件初始化序列，替代代码未能完全模拟
- 某些固件在中断向量表配置方面与仿真环境不兼容

4.4.2 运行稳定性评估

对于启动成功的固件，进一步评估其在仿真环境中的运行稳定性。通过在仿真环境中运行固件 10 分钟，记录崩溃次数和正常运行时间，评估稳定性。实验结果如表 4.6所示。

实验结果表明，70% 的固件在 10 分钟的测试中未发生崩溃，表现出较高的稳定性；30% 的固件发生了 1-2 次崩溃，稳定性为中等。崩溃主要发生在复杂外设（如 ETH）的模拟过程中。

表 4.6 固件仿真运行稳定性

固件编号	运行时间（分钟）	崩溃次数	稳定性评价
F01	10	0	高
F02	10	0	高
F03	10	1	中
F06	10	0	高
F07	10	0	高
F08	10	2	中
F11	10	0	高
F13	10	1	中
F17	10	0	高
F18	10	1	中

4.4.3 动态反馈闭环效果评估

为了评估动态反馈闭环机制的效果，本研究对比了有无反馈闭环情况下固件的运行稳定性。实验结果表明，引入反馈闭环后，固件的平均崩溃次数从 2.3 次降低到 0.8 次，稳定性显著提升。

反馈闭环通过以下机制提升仿真质量：

- 当检测到程序卡死时，自动分析卡死原因，反馈给 LLM 重新生成相关驱动函数
- 当检测到断言失败时，分析失败条件，调整替代代码的逻辑
- 当检测到异常中断时，优化中断处理的模拟逻辑

4.5 与现有方法的对比

本节将本文方法与现有固件仿真方法进行对比，重点分析与本文采用相似思路（驱动替换/半虚拟化）但缺乏大模型自动化机制的工具，以评估本文方法的创新点和优势。

4.5.1 对比方法

本研究选择以下两种具有代表性的方法作为对比，它们与本文方法在核心思想上相似，但自动化程度和实现机制存在显著差异：

（1）**Halucinator**：基于驱动替换的固件仿真框架，通过替换目标固件中的 HAL 函数调用，实现对硬件依赖的解耦。用户需要手工编写每个 HAL 函数的替代实现，这些实现通常返回模拟数据或执行简单的状态转换。Halucinator 的优势在于概念清晰、可控性强，但其局限性在于需要大量的人工分析和编码工作，每个新固件都需要重新编写对应的替换层代码。

（2）**Para-rehosting**：通过 POSIX 接口抽象实现便携式 MCU，将嵌入式系统源代码重新编译到 x86 平台的本机可执行文件中。该方法允许利用 x86 平台上丰富的测试工具进行固件分析。与本文方法相比，Para-rehosting 的核心差异在于需要获取固件样本的源代码，这在第三方的固件分析工作中往往是困难甚至完全不可行的。此外，Para-rehosting 需要实现完整的便携式 MCU 抽象层，工作量大且难以覆盖所有外设功能。

4.5.2 对比实验设计

从测试集中选择 10 个代表性固件（涵盖 STM32F4、STM32F429 和 i.MX RT1052 平台），分别使用三种方法进行仿真，对比各项指标。对比维度包括：准备时间（替换层编写/移植）、自动化程度、编译成功率、启动成功率、可扩展性。

对于 Halucinator，本研究模拟真实使用场景：由熟悉嵌入式开发的工程师根据固件代码和硬件规格，手工编写必需的替换代码，记录所需时间。对于 Para-rehosting，假设源代码可获得的情况下，工程师实现必要的 POSIX 抽象层和外设模拟，记录移植所需时间。对于本文方法，使用完整的自动化流程运行，记录端到端执行时间。

4.5.3 对比结果

对比实验结果如表 4.7所示。

实验结果表明，本文方法在自动化程度和效率方面显著优于 Halucinator 和 Para-rehosting，同时保持了相近的编译成功率和更高的启动成功率。具体分析如下：

表 4.7 与现有方法的对比结果

方法	准备时间	自动化程度	编译成功率	启动成功率	可扩展性
Halucinator	3-6 小时/固件	低（手工编写）	100%	78%	低
Para-rehosting	8-12 小时/固件	低（手工移植）	100%	80%	低
本文方法	**3-5 分钟/固件**	**高（完全自动化）**	**95.3%**	**85.0%**	**高**

（1）准备时间：Halucinator 和 Para-rehosting 的准备时间主要消耗在手工编写或移植代码上。Halucinator 需要为每个 HAL 函数编写替换实现，每个固件需要 3-6 小时；Para-rehosting 需要实现完整的便携式 MCU 抽象层，并移植到 x86 平台，每个固件需要 8-12 小时。本文方法通过 LLM 自动生成替换代码，将准备时间降低到分钟级别，效率提升达两个数量级。

（2）自动化程度：Halucinator 和 Para-rehosting 虽然提供了框架支持，但核心替换逻辑或移植工作需要人工完成，难以规模化。本文方法实现了从驱动函数识别到代码生成的全流程自动化，无需人工干预即可处理不同平台和不同类型的固件。

（3）编译成功率：Halucinator 和 Para-rehosting 由于人工编写或移植代码，在源代码和硬件规格充分理解的情况下，可以确保 100% 编译成功。本文方法通过编译修复循环，最终编译成功率达到 95.3%，虽然略低于手工方法，但考虑到自动化优势，这一性能完全可接受。

（4）启动成功率：本文方法的启动成功率为 85.0%，高于 Halucinator（78%）和 Para-rehosting（80%）。这主要得益于 LLM 对驱动函数语义的深入理解，生成的替代代码在语义正确性上优于人工快速编写的代码。Halucinator 和 Para-rehosting 的手工替换/移植往往倾向于简化实现（如直接返回固定值或简化 POSIX 调用），容易导致固件运行时出现意外行为。

（5）可扩展性：Halucinator 和 Para-rehosting 的可扩展性受限于人力资源，处理新固件或新平台需要大量编码或移植工作。此外，Para-rehosting 受限于源代码的获取，在实际第三方固件分析场景中应用受限。本文方法具有良好的可扩展性，新增平台仅需配置编译工具链和内存映射，代码生成逻辑具有泛化能力，且不依赖源代码。

4.5.4 讨论

通过对比 Halucinator 和 Para-rehosting，本文方法的优势主要体现在以下方面：

（1）自动化程度高：从静态分析识别驱动函数，到 LLM 生成替代代码，再到编译修复和动态验证，整个流程高度自动化，无需人工干预。相比之下，Halucinator 和 Para-rehosting 虽然概念清晰，但核心的替换逻辑或移植工作仍需手工完成，难以实现规模化应用。

（2）语义理解能力强：本文方法利用 LLM 的代码理解能力，能够分析驱动函数的调用模式、数据流向和控制流，生成语义正确的替代代码。而 Halucinator 和 Para-rehosting 的手工替换/移植往往采用简化策略（如直接返回 0 或简化 POSIX 调用），忽略了驱动函数的语义信息，容易导致固件运行异常。

（3）可扩展性和泛化能力好：本文方法的设计具有良好的可扩展性，新增硬件平台仅需配置编译工具链和内存映射，代码生成逻辑具有泛化能力，可应用于不同类型的固件。Halucinator 和 Para-rehosting 的可扩展性受限于人力资源，每个新固件都需要重新编写替换层代码或进行移植。

（4）无需源代码：本文方法直接分析二进制固件（或编译后的 ELF 文件），不依赖源代码的获取。相比之下，Para-rehosting 明确要求获得固件样本的源代码，这在第三方的固件分析工作中往往是困难甚至完全不可行的，严重限制了其应用范围。

（5）迭代优化机制：本文方法建立了完整的动态反馈闭环，通过在仿真环境中执行固件，自动检测问题并触发重新分析和代码生成，实现迭代优化。Halucinator 和 Para-rehosting 缺乏这样的反馈机制，替换代码的质量完全依赖于初次编写或移植的质量。

本文方法的不足主要体现在以下方面：

（1）对 LLM 的依赖：生成代码的质量受限于 LLM 的能力，对于复杂的驱动逻辑或罕见的编程模式，LLM 可能需要多次迭代才能生成正确的代码。相比之下，手工编写的代码（如 Halucinator 和 Para-rehosting 方式）在工程师充分理解后，可以一次到位。

（2）编译成功率略低：本文方法的编译成功率为 95.3%，略低于 Halucinator 和

Para-rehosting 的 100%。虽然通过编译修复循环可以弥补这一差距，但对于追求绝对确定性的场景，手工编写仍有优势。

（3）冷启动成本：本文方法需要预先训练或配置 CodeQL 查询规则和 LLM 提示词模板，对于全新的固件或全新的外设类型，可能需要一定的适配成本。Halucinator 和 Para-rehosting 没有这一冷启动成本，但每次使用的人力成本更高。

总体而言，本文方法在自动化程度、效率和可扩展性方面显著优于 Halucinator 和 Para-rehosting，适合需要大规模处理多种固件的场景。对于单个固件、拥有源代码且对替换代码质量有极高要求的场景，手工移植方法仍有其价值。

4.6 模糊测试应用验证

本节验证本文方法在支持模糊测试方面的有效性。

4.6.1 实验设计

从测试集中选择 5 个包含网络协议栈的固件（F02、F06、F11、F17、F19），使用本文方法进行驱动替换后，在仿真环境中运行 AFL++ 进行模糊测试。测试时间为 24 小时，统计基本块覆盖率和执行速度。

4.6.2 实验结果

模糊测试实验结果如表 4.8所示。

表 4.8 模糊测试实验结果

固件编号	基本块覆盖率	执行速度（exec/s）	发现 Crash
F02	67.3%	2,450	3
F06	72.1%	1,980	5
F11	69.8%	2,120	4
F17	74.5%	1,850	6
F19	71.2%	2,030	4
平均	**71.0%**	**2,086**	**4.4**

实验结果表明，在本文方法构建的仿真环境中，AFL++ 能够有效地对固件进行模糊测试。平均基本块覆盖率达到 71.0%，平均执行速度为 2086 exec/s，在 24 小时

内平均发现 4.4 个 Crash。这些结果表明，本文方法能够有效支持后续的模糊测试和漏洞挖掘任务。

4.7 本章小结

本章通过系统性的实验验证了本文方法的有效性。实验结果表明：

- 静态分析模块能够准确识别固件中的硬件依赖代码，平均准确率和召回率均超过 90%
- LLM 驱动替换模块能够生成高质量的替代代码，最终编译成功率达到 95.3%
- 替换后的固件在仿真环境中具有较高的运行稳定性，85.0% 的固件能够正常启动
- 与现有方法相比，本文方法在效率和效果之间取得了良好的平衡
- 本文方法能够有效支持后续的模糊测试和漏洞挖掘任务

这些实验结果充分证明了本文方法的有效性和实用性，为物联网设备固件仿真提供了一种新的技术途径。

5 总结与展望

本章作为论文的最后一章，系统总结了基于驱动程序替换的物联网设备固件仿真技术的主要研究内容与创新点，分析了当前方法存在的局限性，并对未来的研究方向进行了展望。

5.1 工作总结

本文针对物联网设备固件仿真中的硬件依赖问题，提出了一种基于驱动程序替换的固件仿真方法。该方法的核心思想是通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型理解驱动函数语义并生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量，从而在无需手动建立外设模型的情况下实现固件在仿真环境中的正常运行。

本研究首先设计并实现了面向嵌入式固件的硬件依赖代码自动识别模块。该模块通过 MMIO 模式识别技术定位固件中的外设寄存器访问点，通过数据流追踪技术分析寄存器访问与函数之间的依赖关系，通过递归归纳技术确定驱动函数的边界。该模块能够处理多种嵌入式平台（STM32、NXP 等）和操作系统（FreeRTOS、RT-Thread、裸机）的固件，构建可查询的驱动知识库，为后续的驱动替换提供准确的输入。

在此基础上，本研究构建了基于大语言模型的驱动函数分析与替换模块。该模块利用大语言模型的代码理解能力，对识别出的驱动函数进行语义分析，理解其与硬件的交互逻辑，并自动生成可在仿真环境中运行的替代实现。通过设计结构化的提示词模板和函数分类策略，该模块能够处理不同类型的驱动函数（初始化、数据传输、中断处理等），并通过编译修复循环保障生成代码的可编译性。实验表明，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为。

最后，本研究引入了动态测试反馈闭环机制。当固件在仿真环境中运行时，该机制能够捕获运行时异常（如程序卡死、外设访问超时、断言失败等），分析异常原因，并将分析结果反馈给驱动替换模块以迭代优化替代代码。这种”生成—测试—反

馈—修正”的闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖。实验结果表明，该反馈机制能够在有限的迭代轮次内显著提高固件在仿真环境中的稳定性和运行覆盖率。

综上所述，本研究提出的方法为物联网设备固件仿真提供了一种新的技术途径。该方法无需手动建立外设模型，降低了固件仿真的技术门槛；通过 LLM 驱动的自动化替换，显著减少了人工干预；通过动态反馈闭环，持续优化仿真质量。这些特性使得该方法具有良好的实用性和可扩展性，为后续的模糊测试和漏洞挖掘奠定了基础。

5.2 论文创新点

本研究在物联网设备固件仿真领域取得了以下创新性成果：

（1）硬件依赖代码的自动识别与定位

本研究提出了一种面向嵌入式固件的硬件依赖自动识别方法。与现有依赖人工标注或有限规则推断的方法不同，该方法结合了 MMIO 模式识别、数据流追踪与递归归纳三种技术，能够从固件二进制或源码中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界。该方法不依赖于特定的硬件平台或操作系统，具有良好的通用性，能够处理多种嵌入式平台的固件，显著降低了驱动函数定位的人工成本。

（2）基于大语言模型的驱动函数语义理解与代码生成

本研究首次将大语言模型应用于嵌入式固件的驱动函数替换任务。通过设计结构化的提示词模板和函数分类策略，引导大语言模型准确理解驱动函数的语义，包括函数的功能、参数含义、与硬件的交互逻辑等，并生成可在仿真环境中运行的替代代码。与现有基于预设返回值的简单 Hook 方法相比，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为，为后续的模糊测试提供了更真实的仿真环境。

（3）动态测试反馈闭环驱动的仿真质量优化

本研究构建了”分析—生成—测试—优化”的闭环流程，利用运行证据驱动替换策略的迭代优化。当固件在仿真环境中运行出现异常时，系统能够自动分析异常原

因（如状态寄存器值不合理、时序逻辑错误等），并将分析结果反馈给驱动替换模块，指导后续的代码生成。这种闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖，提高了方法的自动化程度和实用性。

5.3 论文贡献

本研究在理论、技术和实践三个层面做出了以下贡献：

理论贡献：本研究提出了结合大语言模型的自动化固件仿真方法，将大模型的代码理解与生成能力应用于驱动函数替换任务。与现有依赖人工编写替换代码或预设返回值的驱动替换方法不同，本研究探索了利用 LLM 自动理解驱动函数语义、自动生成替代代码的技术路径。该方法论证明了 LLM 在嵌入式固件仿真领域的应用潜力，为固件仿真研究的自动化和智能化提供了新的思路。

技术贡献：本研究实现了完整的工具链，包括静态分析模块（MMIO 模式识别、数据流追踪、递归归纳）、LLM 驱动的驱动替换模块（函数分类、提示词设计、编译修复）、以及动态测试反馈模块（异常捕获、原因分析、策略调整）。这些技术组件相互配合，形成了一套完整的自动化固件仿真解决方案。

实践贡献：本研究在多种嵌入式平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）上验证了方法的有效性，展示了该方法在实际固件仿真任务中的可行性和实用性。实验结果表明，该方法能够支持多种类型的固件在仿真环境中正常运行，为固件安全测试提供了新的技术途径，对于提升物联网设备的安全性具有实际意义。

5.4 局限性与不足

尽管本研究取得了一定的成果，但仍存在以下局限性和不足：

（1）对固件源码的依赖

当前方法主要针对有源码的固件进行分析和替换。对于闭源固件，由于缺乏源码信息，静态分析模块难以准确识别驱动函数的语义，LLM 生成的替代代码质量也会受到影响。虽然理论上可以对二进制固件进行反编译和分析，但反编译结果的准

确性和可读性限制了方法的效果。如何在没有源码的情况下进行高质量的驱动替换，是未来需要解决的问题。

（2）大语言模型的成本和效率问题

大语言模型调用存在显著的成本和时间开销。对于包含大量驱动函数的固件，进行完整的分析和替换可能需要调用 LLM 数十甚至上百次，这不仅带来经济成本，也影响处理效率。此外，LLM 生成代码的质量存在不确定性，有时需要多次迭代才能获得满意的结果。如何降低对 LLM 的依赖，提高处理效率，是实际应用中需要考虑的问题。

（3）平台和操作系统覆盖范围有限

目前方法支持的平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）仍然有限。物联网设备生态复杂，存在大量其他平台（如 ESP32、Nordic nRF 系列等）和操作系统（如 Zephyr、Mbed OS 等）。不同平台的外设实现和驱动接口存在差异，需要针对性地扩展静态分析规则和提示词模板。扩展方法的覆盖范围，使其能够支持更多的嵌入式平台，是未来工作的重要方向。

（4）复杂外设行为的模拟精度

对于一些复杂的外设（如网络控制器、USB 控制器等），其内部状态机逻辑复杂，状态转换涉及多个寄存器的协调操作和时序约束。当前方法生成的替代代码可能无法完全准确地模拟这些复杂行为，导致固件在某些场景下的运行结果与真实硬件存在偏差。如何提高对复杂外设行为的模拟精度，是提升仿真质量的关键。

5.5 未来工作展望

针对上述局限性，未来的研究可以从以下几个方向展开：

（1）支持无源码固件的驱动替换

研究如何在缺乏源码的情况下进行驱动函数的识别和替换。可以探索基于二进制分析的驱动函数定位技术，利用反编译工具（如 Ghidra、IDA Pro）提取函数的中间表示，结合模式匹配和机器学习方法识别驱动函数。对于 LLM 输入，可以尝试使用反编译后的伪代码或控制流图作为输入，引导模型生成替代代码。此外，还可以

研究基于动态分析的驱动行为学习方法，通过观察固件在真实硬件上的执行轨迹，推断驱动函数的语义和行为模式。

（2）降低大语言模型的调用成本

探索降低 LLM 调用成本的多种策略。一方面，可以通过模型微调（Fine-tuning）技术，在嵌入式驱动代码数据集上训练专门的代码生成模型，使其更适合驱动替换任务，从而减少对通用大模型的依赖。另一方面，可以设计更高效的提示词策略，通过 Few-shot Learning 或检索增强生成（RAG）技术，利用已有的驱动替换案例指导新代码的生成，减少迭代次数。此外，还可以研究代码缓存和复用机制，对于相似功能的驱动函数，直接复用已有的替代代码，避免重复调用 LLM。

（3）扩展平台和操作系统的支持范围

逐步扩展方法对不同嵌入式平台和操作系统的支持。可以采用增量扩展的策略，优先支持市场占有率较高的平台（如 ESP32、nRF52 等）和操作系统（如 Zephyr、Mbed OS 等）。对于新平台的适配，需要收集该平台的固件样本，分析其外设实现特点和驱动接口规范，扩展静态分析规则库和提示词模板库。此外，还可以研究自动化的平台适配技术，通过分析平台的 SDK 和头文件，自动生成适配规则。

（4）提升复杂外设的模拟精度

针对复杂外设的模拟问题，可以研究基于状态机学习的建模方法。通过观察固件与外设的交互序列，自动学习外设的状态机模型，包括状态集合、转移条件和输出行为。对于时序敏感的操作，可以引入时间建模机制，在仿真环境中模拟硬件的时间行为。此外，还可以探索混合仿真策略，对于关键的复杂外设，采用高精度的仿真模型；对于简单的外设，采用 LLM 生成的替代代码，在精度和效率之间取得平衡。

（5）安全性验证与保障

引入形式化验证方法，确保替换代码的安全性和正确性。可以研究基于符号执行的验证技术，对 LLM 生成的替代代码进行自动化的安全性检查，确保其不会引入新的安全漏洞。此外，还可以建立驱动替换代码的测试基准（Test Oracle），通过与真实硬件行为的对比，验证仿真结果的正确性。对于关键应用场景（如工业控制、医疗设备等），需要更严格的验证机制，确保仿真环境的可靠性。

致 谢

本论文的完成离不开许多人的帮助和支持，在此向他们表达我最诚挚的感谢。

首先，我要感谢我的导师。在论文选题、研究方法、实验设计等各个方面，导师都给予了我悉心的指导和宝贵的建议。导师严谨的学术态度、渊博的专业知识和敏锐的学术洞察力，使我受益匪浅。

感谢实验室的各位同学。在研究过程中，我们相互讨论、相互帮助，共同解决了很多难题。特别感谢在实验环境搭建、测试用例设计等方面给予我帮助的同学们。

感谢我的家人。他们一直以来的理解、支持和鼓励是我完成学业的坚强后盾。

最后，感谢所有在论文评审和答辩过程中提出宝贵意见的专家学者，你们的建议使本论文得以完善。

作者

于华中科技大学

参考文献

[1] GSMA. The Mobile Economy 2020. https://www.gsma.com/mobileeconomy/, 2020

[2] Huawei. Huawei LiteOS. https://www.huawei.com/en/products/iot/operating-system

[3] Amazon Web Services. FreeRTOS - Real-time operating system for microcontrollers. https://www.freertos.org/

[4] Linux Foundation. Zephyr Project - Real-time operating system for IoT. https://www.zephyrproject.org/

[5] 杨毅宇, 周威, 赵尚儒, 刘聪, 张宇辉, 王鹤, et al. 物联网安全研究综述: 威胁, 检测与防御. 通信学报, 2021, 42(8): 18

[6] Manos Antonakakis, Tim April, Michael Bailey, Mathew Bernhard, Elie Bursztein, Jaime Cochran, et al. Analysis of the Mirai Botnet. In: USENIX Security Symposium, 2018

[7] Daming D Chen, Manuel Egele, Maverick Woo, David Brumley. Firmadyne: Towards Automated Dynamic Firmware Analysis. In: IEEE Symposium on Security and Privacy, 2016: 1–16

[8] Tobias Scharnowski, Nils Bars, Moritz Schloegel, Eric Gustafson, Marius Muench, Giovanni Vigna, et al. Fuzzware: Using Precise MMIO Modeling for Effective Firmware Fuzzing. In: 31st USENIX Security Symposium, 2022

[9] Fabrice Bellard, et al. QEMU, the FAST! processor emulator. https://www.qemu.org/

[10] Musard Kamilov, et al. Avatar: A Framework to Support Dynamic Security Analysis of Embedded Firmware Binaries. In: NDSS, 2015

[11] Marius Muench, Jan Stijohann, Fabian Kargl, Aurélien Francillon, Davide Balzarotti. Avatar^2: A Multi-target Orchestration Platform. In: Workshop on Binary Analysis Research, 2018

[12] Eric Gustafson, Marius Muench, Chad Spensky, Tobias Scharnowski, Paul Grosen, Stephen Mavidis, et al. Pretender: High-Fidelity Peripheral Emulation for Firmware. In: USENIX Security Symposium, 2019

[13] Bo Feng, Alejandro Mera, Long Lu. P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling. In: Proceedings of Usenix Security Symposium, 2020

[14] Chen Cao, Le Guan, Peng Cui, Limin Cheng, Mingyu Zhu. Laelaps: An Efficient and Fast Dynamic Analysis Tool for IoT Firmware. In: IEEE International Conference on Embedded Software and Systems, 2021

[15] Wei Zhou, et al. uEmu: A Framework for Firmware Emulation and Vulnerability Discovery. In: ACM CCS, 2021

[16] Wenqiang Li, Libo Li, Shenglin Zhang, Tao Xie, Lei Ma, et al. Para-rehosting: Automatic Payload Generation for Embedded Inter-Process Communication. In: USENIX Security Symposium, 2022

[17] Abraham A Clements, Eric Gustafson, Tobias Scharnowski, Paul Grosen, David Fritz, Christopher Kruegel, et al. HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation. In: 29th USENIX Security Symposium, 2020: 1–18

[18] Jiongyi Chen, Wenrui Diao, Qingchuan Zhao, Chaoshun Zuo, Zhiqiang Lin, XiaoFeng Guo, et al. IoTFuzzer: Discovering Memory Corruptions in IoT Through App-based Fuzzing. In: NDSS, 2018

[19] Yan Shoshitaishvili, Ruoyu Wang, Christophe Hauser, Christopher Kruegel, Giovanni Vigna. Firmalice: Automatic Detection of Authentication Bypass Vulnerabilities in Binary Firmware. In: NDSS, 2015

[20] OpenAI. GPT-4 Technical Report. arXiv preprint arXiv:2303.08774, 2023

[21] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, et al. Code Llama: Open Foundation Models for Code. arXiv preprint arXiv:2308.12950, 2023

[22] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, et al. StarCoder: May the Source Be with You! arXiv preprint arXiv:2305.06161, 2023

[23] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, et al. Finetuned Language Models Are Zero-Shot Learners. arXiv preprint arXiv:2109.01652, 2022

[24] Weize Wang, Qipeng Chen, Zihao Zhang, Yang Li, Xin Cui, Jing Chen, et al. DolphCoder: Echo-Locating Code with Diverse and Multi-Objective Instruction Tuning. arXiv preprint, 2024

[25] Tianyu Zhang, et al. CodeLSI: Leveraging Foundation Models for Automated Code Generation with Low-Rank Optimization and Domain-Specific Instruction Tuning. arXiv preprint, 2024

[26] Xinyun Chen, Maxwell Lin, Dale Schuurmans, Quoc Le. Teaching Large Language Models to Self-Debug. arXiv preprint arXiv:2304.05128, 2023

[27] Kechi Liu, Shusheng Zhang, Ge Li, Zhi Jin, Li Zhao, Yang Li. Tuning Models of Code with Compiler-Generated Reinforcement Learning Feedback. arXiv preprint, 2024

[28] Yuxiang Zhang, et al. CodeDPO: Aligning Code Models with Self-Generated and Verified Source Code. arXiv preprint, 2024

[29] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv preprint arXiv:2201.11903, 2022

[30] Kechi Zhang, et al. MoTCoder: Elevating Large Language Models with Modular of Thought for Challenging Programming Tasks. arXiv preprint, 2024

[31] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, et al. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv preprint arXiv:2302.02531, 2023

[32] Wei Chen, et al. MCCoder: Streamlining Motion Control with LLM-Assisted Code Generation and Rigorous Verification. arXiv preprint, 2024

[33] Zhi Li, et al. RGD: Multi-LLM Based Agent Debugger via Refinement and Generation Guidance. arXiv preprint, 2024

[34] Hao Li, et al. GPIoT: Tailoring Small Language Models for IoT Program Synthesis and Development. arXiv preprint, 2024

[35] Simon Müller, et al. Towards Specification-Driven LLM-Based Generation of Embedded Automotive Software. arXiv preprint, 2024

[36] Yu Chen, et al. Improving LLM-Assisted Secure Code Generation through Retrieval-Augmented Generation and Multi-Tool Feedback. arXiv preprint, 2024

[37] Tianyu Wang, et al. Agents4PLC: Automating Closed-Loop PLC Code Generation and Verification in Industrial Control Systems Using LLM-Based Agents. arXiv preprint, 2024

[38] RT-Thread Community. RT-Thread: An Open Source Embedded Real-Time Operating System. https://www.rt-thread.org/

[39] Michal Zalewski. American fuzzy lop (AFL). https://lcamtuf.coredump.cx/afl/

攻读硕士学位期间发表的学术论文

1)（待补充）